1. INTRODUCTION

1.1 About the Company — Antier Solutions

Antier Solutions is Enterprise-grade Web3 and blockchain development company with a global footprint spanning North America, Europe, and Asia-Pacific. Founded with a mission to "decentralize the world," Antier helps businesses across industries adopt decentralized technologies to stay ahead of rapid technological change and seize opportunities in the Web3 economy. Antier offers a comprehensive portfolio of blockchain engineering services, including custom blockchain development, Layer 1 and Layer 2 network solutions (Substrate, OP Stack, Arbitrum Orbit, Polygon zkEVM, zkSync Hyperchains), decentralized exchange (DEX) development, DeFi protocol engineering, NFT marketplace development, DePIN (Decentralized Physical Infrastructure Networks) development, cross-chain interoperability solutions, and enterprise-grade identity management systems.

The company's engineering teams work across multiple blockchain frameworks and networks Ethereum, Solana, BSC, Cardano, Polkadot, Cosmos SDK, Avalanche, SUI and have built production-grade blockchain solutions for clients across verticals including financial services, supply chain, healthcare, entertainment, and real estate. Antier's offshore capacity center model enables global businesses to build dedicated blockchain engineering teams with the expertise and velocity typically available only to large technology enterprises.

Antier Solutions is ISO-certified and employs a team of 200+ blockchain engineers, smart contract developers, and Web3 architects. The company has successfully delivered over 500 blockchain projects and maintains a strong focus on security, scalability, and regulatory compliance across all of its development engagements. Its client base spans early-stage blockchain startups as well as established institutional clients requiring enterprise-grade decentralized infrastructure. Beyond development services, Antier Solutions actively contributes to blockchain ecosystem consulting and strategic advisory for enterprises transitioning into Web3. This includes tokenomics design, blockchain architecture consulting, go-to-market planning for decentralized products, and guidance on compliance frameworks across multiple jurisdictions. Many clients approach Antier not only for technical execution but also for architectural planning, infrastructure selection, and product scalability assessments before entering implementation. This consulting-first approach enables organizations to identify the most appropriate blockchain model—public, private, permissioned, or hybrid—based on operational requirements, governance needs, and long-term product objectives.

Antier also follows a highly collaborative engineering workflow centered around agile development and continuous client engagement. Project teams are structured with solution architects, blockchain engineers, backend developers, frontend engineers, DevOps specialists, QA teams, and project managers working in iterative sprint cycles. This structure enables frequent release cycles, rapid feature validation, and continuous testing throughout the development lifecycle. Security remains a critical organizational focus, particularly for financial and tokenized asset infrastructure, where Antier incorporates code review practices, automated vulnerability scanning, penetration testing coordination, and third-party smart contract audits prior to deployment.

A defining organizational advantage at Antier is its strong specialization in emerging blockchain infrastructure. While many software companies offer blockchain services as an extension of traditional development, Antier operates with blockchain engineering as its core business vertical. This specialization allows the company to maintain deep expertise in protocol-level infrastructure, cryptographic systems, decentralized finance primitives, token standards, governance architecture, and multi-chain interoperability. For internship trainees and engineers, this creates an environment where development work involves real blockchain infrastructure rather than simulated learning projects,

1.2 Projects and Technologies at Antier Solutions

Antier Solutions operates across several key practice areas, each serving different aspects of the Web3 development lifecycle:

- **DeFi and Exchange Platforms:** Antier designs and deploys decentralized exchange protocols, automated market makers (AMMs), liquidity pool mechanisms, yield farming contracts, and staking platforms including projects with custom bonding curve pricing and algorithmic liquidity management.
- **Real-World Asset (RWA) Tokenization:** Antier builds blockchain infrastructure for tokenizing real-world assets including real estate, commodities, and structured financial products, enabling fractional ownership and secondary trading of traditionally illiquid assets.
- **Layer 2 and Custom Chain Development:** Antier engineers custom Layer 2 rollup networks using the OP Stack, Arbitrum Orbit, and zkSync Hyperchains frameworks enabling clients to deploy their own EVM-compatible blockchain with custom fee structures, validator sets, and governance models.
- **Enterprise Blockchain Solutions:** Antier builds permissioned and hybrid blockchain networks for enterprise clients in supply chain, healthcare, identity

management, and finance, using Hyperledger Fabric, Cosmos SDK, and Polkadot Substrate.

- **Smart Contract Development and Auditing:** All Antier projects include Solidity/Rust smart contract development with comprehensive unit and integration testing via Hardhat and Foundry frameworks, and security auditing using industry-standard methodologies.

The Libertum project — the subject of this internship report — was developed under Antier Solutions' DeFi and RWA Tokenization practice. It represents a flagship-level engagement combining multiple of Antier's core competencies: EVM-compatible Layer 2 blockchain integration, complex DeFi mechanics (bonding curve DEX, P2P trading engine), enterprise-grade microservices backend, and regulatory compliance infrastructure.

1.3 Overview of the Internship Project — Libertum

Libertum is a production-deployed, full-stack decentralized real estate tokenization ecosystem built on the Base blockchain — an Ethereum-compatible Layer 2 network developed by Coinbase using the OP Stack. It is not a prototype or proof-of-concept: it is a live system serving real users with real transactions, secured by audited smart contracts, and governed by integrated regulatory compliance. The platform was engineered entirely by the team at Antier Solutions.

At its architectural core, Libertum brings together eight independent, containerized backend microservices that work in concert to deliver the complete investor lifecycle — from user registration and identity verification, through fractional real estate investment and secondary P2P trading, all the way to on-chain token settlement, blockchain event tracking, and automated transactional email notifications. Every service is independently deployable, independently scalable, and communicates exclusively through well-defined API contracts and Apache Kafka event streams. A major distinguishing factor of Libertum is its hybrid interaction model between blockchain execution and off-chain backend orchestration. While asset ownership and settlement are enforced on-chain through smart contracts, operational workflows such as KYC verification, email communication, dashboard aggregation, analytics, and administrative controls are processed off-chain through backend services. This separation ensures that blockchain is used where immutability and transparency are essential, while performance-sensitive business operations remain scalable and cost-efficient in the traditional backend environment.

The investor experience on Libertum follows a complete digital investment lifecycle. A new user registers, completes identity verification, accesses tokenized asset listings, purchases fractional shares, monitors holdings through portfolio dashboards, and can later trade those

holdings through the secondary P2P market. Every blockchain action generates corresponding event updates that synchronize with internal services and update user-facing dashboards in near real time. This architecture creates a seamless experience where blockchain transactions remain technically decentralized while still presenting the responsiveness expected from modern fintech platforms.

Another notable aspect of Libertum is operational transparency. Ownership records are publicly verifiable on-chain, transaction history is auditable, and token movement is recorded across both blockchain and internal service layers. This improves trust for investors while simplifying administrative oversight. Since real estate transactions traditionally involve multiple intermediaries and delayed paperwork, Libertum demonstrates how blockchain-based automation can significantly improve both execution speed and investor accessibility.

The platform is built on three foundational engineering principles:

- **Decentralization:** Smart contracts deployed on the Base blockchain govern all financial operations — token minting, escrow-based P2P settlement, bonding curve pricing, staking, and dividend distribution. No single party holds user funds or controls trade settlement; the contract code is the authority. No single party directly controls investor funds, modifies transaction execution, or manually approves settlements once smart contract conditions are met. This significantly reduces dependency on intermediaries traditionally involved in real estate investment such as brokers, clearing entities, or centralized custodians.
- **Compliance:** Every user must pass KYC (Know Your Customer) identity verification for individual investors, or KYB (Know Your Business) verification for institutional entities, before accessing any investment or trading feature. Transfer agents independently authorize all token transfers, maintaining a compliant and auditable cap table at all times. The platform continuously maintains a compliant cap table and transaction history, which is especially important for tokenized securities and fractional real estate ownership. By embedding compliance into the technical workflow, Libertum balances blockchain decentralization with practical regulatory obligations required in real-world financial markets
- **Scalability:** Microservice isolation ensures that high-traffic components — the P2P order engine, the blockchain event listener, the bonding DEX — can be scaled independently without impacting the performance of compliance, ledger, or administrative services. Kafka-based async messaging decouples all services from each other.

1.4 Importance in the Current Industry Landscape

The global real estate market is valued at over $326 trillion — the single largest asset class on earth, exceeding global equity and bond markets combined. Yet structural barriers have historically limited participation to high-net-worth individuals and institutional investors. The emergence of blockchain-based real-world asset (RWA) tokenization is changing this landscape fundamentally.

The global RWA tokenization market is projected to reach $16 trillion by 2030 (Boston Consulting Group, 2022). Major institutional players including BlackRock, JPMorgan, and Goldman Sachs have launched tokenized asset products on public and permissioned blockchains. Regulatory frameworks for security tokens are maturing in the United States (SEC Regulation D exemptions), European Union (MiCA), and Singapore (MAS framework). Ethereum-compatible Layer 2 networks like Base have reduced transaction costs to sub-cent levels, making on-chain financial interactions economically viable for retail users. Libertum is positioned at the precise convergence of these macro-level trends. As a production-deployed platform with real users and real transactions, it demonstrates that decentralized real estate investment is not a theoretical future state — it is a commercially operational reality. The engineering architecture developed during this internship serves as a transferable blueprint for the broader RWA tokenization wave projected to reshape global capital markets over the next decade. Another important macroeconomic factor supporting tokenized real-world assets is increasing institutional demand for alternative liquidity models. Traditional private market assets—including real estate—typically involve long settlement cycles and limited exit opportunities. Tokenization introduces programmable ownership structures that can significantly reduce settlement friction while improving capital accessibility. This shift has accelerated industry interest in blockchain infrastructure capable of supporting regulated ownership at scale.

The emergence of Ethereum Layer 2 ecosystems has also played a direct role in enabling platforms like Libertum. Historically, Ethereum mainnet transaction fees made frequent financial interactions expensive for smaller investors. Layer 2 networks such as Base address this limitation through rollup-based scaling and lower-cost execution, making high-frequency on-chain portfolio activity economically practical. This infrastructure improvement transforms blockchain from a niche settlement layer into a viable production foundation for mainstream financial products. From an enterprise technology perspective, Libertum also reflects the broader shift toward composable financial infrastructure. Tokenized assets can integrate with wallets, exchanges, compliance providers, analytics tools, and decentralized applications through shared blockchain standards.

1.5 Objectives and Goals of the Project

The overarching objective of the Libertum project is to deliver a production-grade, regulation-compliant, decentralized ecosystem that makes real estate investment accessible, liquid, and transparently priced for a global retail audience. Specific technical and commercial objectives are detailed in Section 2. The author's internship contributions focused on:

- Backend and blockchain integration for the P2P Trading Engine — order lifecycle state machine, atomic MongoDB quantity management, Kafka event propagation, and concurrent multi-party email notification dispatch.
- Bonding Curve DEX Service — linear bonding curve price algorithms, 24-hour volume analytics, Etherscan - enriched token holder data, and Big.js precision time-series portfolio value computation.
- Supporting contributions across the Blockchain Event Service (block-number deduplication, batch polling architecture), Notification Microservice (EJS template rendering, multi-tenant branding), and SaaS Backend (Cloudflare domain whitelisting, 4-step product onboarding workflow).

2. OBJECTIVES

2.1 Overall Libertum Platform Goals

The Libertum platform is governed by four foundational engineering and commercial goals that shape every architectural decision across all eight microservices:

- **Decentralized Tokenization:** Design and implement a secure, fractional real estate tokenization ecosystem on the Base blockchain using ERC-20/ERC-1155 standards. Every token represents an auditable, immutable ownership stake in a real-world property, allowing investments as small as a few dollars and unlocking the asset class for global retail participation.
- **Regulatory Compliance:** Enforce strict KYC/KYB identity verification via SumSub integration, and deploy a standalone Transfer Agent microservice to manage compliant cap tables and authorize or reject on-chain token transfer requests — ensuring every transaction meets institutional regulatory standards.
- **Microservices Resilience:** Build a modular, 8-service distributed architecture with no shared databases, ensuring components can scale and update independently. Apache Kafka provides robust, durable inter-service communication with at-least-once delivery guarantees, preventing cascading failures across service boundaries.

• **Real-Time Blockchain Sync:** Implement dual-mode blockchain event monitoring batch PostgreSQL polling combined with block-number-based deduplication to ensure zero transaction loss and provide sub-second downstream data consistency across the admin dashboard, portfolio tracker, and notification service.

2.2 Libertum P2P Trading Engine Goals

The P2P Trading Engine is Libertum's core secondary market mechanism. Its development is guided by four engineering goals that together eliminate the need for any centralized intermediary in the real estate token trading lifecycle:

• **Eliminate Broker Intermediaries:** Develop a secondary market engine where investors trade property tokens directly with counterparties, secured entirely by on-chain smart contract escrow logic. Sellers lock tokens in escrow at listing time; tokens are released only upon verified payment — removing all settlement risk without any trusted third party.

• **Robust Order State Machine:** Programmatically enforce a secure, deterministic trade lifecycle (PENDING → ACTIVE → PAYMENT_PENDING → PAYMENT_CONFIRMED → TOKENS_RELEASED → COMPLETED) that safely bridges off-chain fiat payment confirmation with on-chain token releases. Each state transition is atomic in MongoDB, preventing partial-state corruption across concurrent operations.

• **Real-Time Actionable Feedback:** Push instant WebSocket payloads and concurrent Kafka-driven email notifications to both buyers and sellers at every state transition, delivering a smooth, Web2-comparable trading experience. Promise.all parallel dispatch ensures neither party's notification blocks the other.

• **Automated Synchronization:** Ensure all completed P2P trades sync autonomously to the centralized Admin analytics dashboard via Kafka event streams, while simultaneously computing parallel NAV (Net Asset Value) pricing records for accurate portfolio tracking — maintaining full data consistency across all 8 services without manual reconciliation.

2.3 Libertum Bonding Curve DEX Goals

The Bonding Curve DEX replaces traditional order-book liquidity with a mathematically-governed pricing contract. Its four engineering goals are designed to provide fair, continuous, and analytically rich token markets:

• **Continuous Algorithmic Liquidity:** Guarantee programmatic buying and always selling availability for new property tokens using a linear supply-demand bonding

curve model: Price(supply) = BasePrice × (1 + k × CurrentSupply). This removes the need for traditional order book liquidity providers and guarantees that any user can always transact at the current mathematically-determined price.

• **Transparent Price Discovery:** Implement a resilient pricing formula that naturally rewards early property investors (lower entry price) and creates scarcity-based appreciation as adoption grows, mirroring real-world property value dynamics. Since price is governed entirely by a smart contract function, it cannot be manipulated by any centralized actor.

• **High-Precision Analytics:** Aggregate enterprise-grade DEX metrics locally — including rolling 24-hour volume calculations, interactive time-series portfolio charts, and cumulative value tracking — utilizing Big.js to prevent floating-point drift in all 18-decimal-precision financial computations. This eliminates the class of precision bugs (e.g., 179.99… instead of 180.00) inherent in JavaScript's IEEE 754 arithmetic.

• **Deep On-Chain Enrichment:** Integrate external Etherscan Pro API feeds to fetch real-time unique wallet holder counts per token contract, merging external blockchain state with local MongoDB aggregation data. High-concurrency batching (chunks of 5 with Promise.all parallelization) prevents rate-limit violations while delivering live holder analytics to DEX discovery listings.

2.4 Specific Problems the Project Solves

Libertum solves the major challenges of traditional real estate investment by improving liquidity, reducing high entry barriers, and enabling transparent price discovery. Through blockchain-based fractional ownership and peer-to-peer secondary market trading, investors can buy and sell tokenized real estate more efficiently. The platform also integrates compliance and automated transaction processing, creating a secure and scalable investment ecosystem. It solves several long-standing structural problems associated with traditional real estate investment by combining blockchain-based ownership infrastructure with an automated digital investment ecosystem. Real estate has historically remained one of the least accessible asset classes for retail investors due to high capital requirements, low liquidity, complex legal processes, and limited transparency. Libertum addresses these barriers through tokenization, decentralized trading mechanisms, and compliance-integrated backend architecture.

Table 1: Problem-Solution Mapping for the Libertum Platform

Problem	Libertum Solution
Real estate is illiquid — investors cannot exit positions before property liquidation.	Fractional tokenization enables partial ownership. P2P secondary market allows instant exit at user-defined prices.
High capital entry barriers exclude retail investors.	Token fractions allow minimum investment as small as $50–$100, democratizing access globally.
No transparent or continuous token pricing mechanism.	Bonding Curve DEX provides real-time, supply-driven algorithmic price discovery without intermediaries.
Broker-dependent trading with high intermediary fees.	Automated smart contract escrow and settlement removes all intermediaries from the trading process.
Lack of compliance in DeFi creates regulatory risk.	Integrated KYC/KYB verification layer gates all trading and investment activities via SumSub.
Existing tokenization platforms are single-client and non-scalable.	Multi-tenant SaaS architecture supports white-label deployment for any real estate business

2.5 Target Users

The platform is designed to serve a broad, globally distributed ecosystem of users, ensuring it functions as comprehensive public market infrastructure rather than a restricted closed-loop system:

- **Retail Investors:** Everyday users seeking fractional real estate assets with low capital requirements ($50–$100 minimum) and full access to secondary liquidity via the P2P and Bonding Curve DEX trading engines.
- **Real Estate Developers & Asset Managers:** Property owners leveraging the platform to tokenize physical real estate portfolios, raise capital rapidly across borders, and bypass traditional banking intermediaries and broker commissions.
- **Real Estate Agencies & Brokerages:** Traditional property firms seeking to list exclusive properties on a digital, borderless marketplace, enabling their clients to trade fractions of commercial and residential assets.

• **High-Net-Worth Individuals (HNWIs) & Institutions:** Large-scale buyers seeking transparent, algorithmic price discovery and robust regulatory compliance (KYB verification) before committing bulk capital to tokenized property portfolios.

• **Independent Transfer Agents & Compliance Officers:** Regulatory bodies or third-party compliance officers who utilize the dedicated Transfer Agent portal to monitor cap tables and manually authorize or reject on-chain token transfer requests.

• **Platform Operators & Admins:** Internal employees managing the overarching ecosystem, overseeing KYC approvals, verifying property documentation, triggering token mints, and monitoring global system health via the Kafka-synced Admin Panel.

2.6 Project Goals

• Achieve production deployment on the Base blockchain with live users and real transactions.

• Deliver sub-second P2P order matching with atomic quantity management and race-condition prevention using MongoDB's $inc operator.

• Implement high-precision bonding curve arithmetic using Big.js to prevent floating-point errors in all financial calculations.

• Establish a fully automated CI/CD pipeline with manual approval gates enabling zero-downtime deployments.

• Build a notification system that covers every critical investor lifecycle event with per-tenant branded transactional emails.

• Maintain complete service isolation: no shared databases across any of the 8+ microservices.

3. LITERATURE REVIEW

3.1 Real Estate Tokenization

The concept of representing real-world asset ownership as blockchain tokens was first proposed in academic literature by Yermack (2017), who argued that distributed ledgers could fundamentally transform corporate governance and asset ownership records. The practical implementation of real estate tokenization gained commercial traction with RealT (2019) and Lofty.ai (2021), which demonstrated that fractional real estate ownership via ERC-20 tokens was both technically viable and commercially attractive to retail investors.

However, both platforms suffered from the same critical architectural limitation: no secondary market. Investors could acquire fractional tokens but had no mechanism to exit their positions before the underlying property was liquidated. This replicated the fundamental illiquidity of

traditional real estate within a blockchain wrapper — solving the entry barrier problem while failing to solve the exit liquidity problem.

More recent academic analysis by Chen et al. (2020) in the IEEE Transactions on Engineering Management identified that for tokenized real estate to achieve mainstream adoption, three components are essential: (i) a compliant issuance mechanism, (ii) a reliable secondary trading market, and (iii) transparent, continuous price discovery. Libertum is designed to deliver all three in a single integrated ecosystem.

3.2 Peer-to-Peer Trading Protocols

The foundational work on peer-to-peer trading using blockchain escrow was developed by the OpenSea protocol (2017) in the context of non-fungible tokens (NFTs). OpenSea demonstrated that buyer-seller matching could happen trustlessly on-chain using smart contract escrow — where neither party needs to trust the other directly, as the smart contract acts as a neutral, automated mediator.

The key insight from OpenSea's architecture — adopted and adapted by Libertum — is that a seller can lock assets in a smart contract escrow at listing time, and only release them to the buyer upon confirmed payment. This eliminates settlement risk, the central problem of all trading systems: the risk that one party fulfils their obligation while the other defaults.

Libertum extends this model specifically for security tokens (tokenized real estate) by adding KYC compliance gates, order expiry management with quantity restoration, partial fill support, and a rich multi-state order lifecycle (PENDING → ACTIVE → PAYMENT_PENDING → PAYMENT_CONFIRMED → TOKENS_RELEASED → COMPLETED) that mirrors professional OTC trading platforms.

3.3 Bonding Curve Theory and AMM Design

The mathematical foundation of bonding curves was formalized by Simon de la Rouviere in his 2017 paper "Tokens 2.0: Curved Token Bonding in Curation Markets," which described a pricing mechanism where a token's price is determined entirely by its current circulating supply through a predefined mathematical function.

Bancor Protocol (2017) was the first to implement bonding curves at commercial scale, introducing the concept of a reserve-backed liquidity pool where tokens could always be bought or sold at the current curve price, backed by a collateral reserve. This guaranteed continuous liquidity regardless of market conditions — a property that traditional order books cannot guarantee.

Uniswap V2 (Adams et al., 2020) popularized the constant-product Automated Market Maker (AMM) formula $x \times y = k$, which became the dominant DEX architecture in decentralized finance. While powerful for general-purpose tokens, the constant-product formula has limitations for real estate tokens: it can produce very large price swings at low liquidity levels and does not naturally model the expected appreciation characteristics of real estate assets.

Libertum adapted a linear bonding curve model: $\text{Price}(\text{supply}) = \text{BasePrice} \times (1 + k \times \text{CurrentSupply})$, where k is a curve steepness constant calibrated per token offering. This linear model has two properties desirable for real estate tokens: (i) early investors receive lower entry prices, rewarding early adoption; and (ii) price appreciation mirrors expected real-world property value appreciation as more investors participate.

3.4 Security Token Standards

The regulatory-compliant tokenization of real-world assets requires specialized token standards that go beyond the standard ERC-20 interface. The most significant are:

- **ERC-1404 (2018):** The Simple Restricted Token Standard, which introduces transfer restriction logic directly into the token contract. Tokens implementing ERC-1404 can reject transfers to non-whitelisted addresses, enabling on-chain KYC enforcement. Libertum's transfer agent architecture was directly informed by this standard.
- **ERC-3643 (T-REX Protocol, 2021):** An enterprise-grade security token standard implementing on-chain identity verification through a compliant identity registry. It enforces that tokens can only be held by verified investors, and transfer restrictions are enforced at the smart contract level.
- **ERC-20 / ERC-1155:** Standard fungible and multi-token interfaces used for fractional property tokens. ERC-1155 is particularly relevant for platforms managing multiple property tokens, as it allows a single contract to manage tokens for many different properties.

3.5 Event-Driven Architecture in Blockchain Systems

A fundamental challenge in blockchain-integrated backend systems is the asynchronous nature of blockchain confirmation. When a user initiates a transaction, it does not confirm instantaneously — it is broadcast to the network, included in a block by a miner/validator, and then propagates across the network. This confirmation of latency (typically 2–15 seconds on Base) means that backend systems cannot rely on synchronous transaction confirmation.

Event-driven architecture (EDA), as documented by Fowler (2017) and implemented commercially through platforms like Apache Kafka, provides the ideal solution. Rather than

polling the blockchain for updates, an event-driven backend listens for specific on-chain events (smart contract logs) and reacts to them asynchronously. Kafka's publish-subscribe model ensures that events are reliably delivered to all interested services, with guaranteed at-least-once delivery and configurable consumer group semantics.

Libertum's Blockchain Event Service implements precisely this pattern: it polls the Base blockchain in configurable batch windows, detects relevant smart contract events, and publishes them to Kafka topics consumed by the Marketplace Service, Admin Service, and Notification Service. Block-number-based deduplication in PostgreSQL ensures that service restarts do not result in duplicate event processing.

3.6 Smart Contract Escrow and Settlement Automation

Smart contract escrow mechanisms are central to blockchain-based trading systems because they eliminate the need for third-party intermediaries during settlement. Traditional financial settlement often depends on clearing houses or custodians to ensure both buyer and seller fulfil their obligations. This introduces delays, manual verification, and settlement risk.

Ethereum introduced programmable smart contracts that made automated escrow practical at scale. Once predefined conditions are met, smart contracts can execute asset transfer automatically without requiring human approval. This trustless execution model became foundational to decentralized exchanges and peer-to-peer asset trading platforms.

In Libertum, escrow contracts are used to lock tokenized real estate assets at listing time. When an investor places a sell order, the token quantity is transferred into the escrow smart contract. The seller no longer directly controls those tokens during the order lifecycle, ensuring they cannot be sold twice or withdrawn after being listed. Once payment

3.7 Blockchain Infrastructure and Base Network Integration

Choosing the underlying blockchain infrastructure is critical for tokenized asset platforms because transaction cost, speed, and scalability directly affect user adoption.

Ethereum remains the dominant smart contract ecosystem, but high gas fees and network congestion create friction for retail transactions. Layer-2 networks were introduced to solve these limitations by processing transactions off-chain while inheriting Ethereum’s security.

Base, launched by Coinbase in 2023, is an Ethereum Layer-2 network built using the OP Stack. It offers significantly lower transaction fees and faster confirmation compared with Ethereum mainnet while maintaining EVM compatibility.

3.8 Wallet Integration and Digital Asset Custody

Digital asset ownership in blockchain ecosystems depends on secure wallet infrastructure. A wallet serves as the investor's blockchain identity and provides access to token balances, transaction approvals, and asset transfers. In tokenized asset platforms, wallet integration must balance usability with institutional-grade security.

MetaMask, Coinbase Wallet, and WalletConnect significantly expanded blockchain adoption by simplifying wallet connectivity across decentralized applications. However, for financial assets such as tokenized real estate, wallet infrastructure must also support compliance monitoring, transaction traceability, and secure asset custody.

Libertum integrates wallet-based authentication and transaction approval directly into its investor workflow. Investors connect their wallets to access offerings, purchase tokens, and participate in peer-to-peer trading. Ownership of security tokens is reflected transparently on-chain, while wallet signatures ensure that all investment actions are cryptographically authorized by the investor.

3.9 Gap Analysis: Existing Solutions

Table 2: Comparative Analysis of Existing Real Estate Tokenization Platforms

Platform	Tokenization	P2P Market	Bonding Curve	KYC/KYB	SaaS	Live
RealT	✓	✗	✗	✓	✗	✓
Lofty.ai	✓	✗	✗	✓	✗	✓
Uniswap V2	✗	✗	✓	✗	✗	✓
OpenSea	✗	✓	✗	✗	✗	✓

As demonstrated in Table 2, no existing platform combines all six capabilities — tokenization, P2P trading, bonding curve DEX, KYC/KYB compliance, multi-tenant SaaS support, and live production deployment — in a single ecosystem. This multi-dimensional gap is precisely the innovation that Libertum addresses.

4. PROBLEM STATEMENT

Traditional real estate investment suffers from three fundamental, deeply entrenched structural problems that have persisted for decades despite advancements in financial technology. These problems are not superficial inconveniences — they are systemic barriers that prevent approximately 4.7 billion potential retail investors from accessing one of the world's highest-performing asset classes.

4.1 Problem 1: Illiquidity

A property cannot be partially sold. An investor who owns 100% of a residential apartment cannot exit 20% of their investment by selling a proportional share — they must either hold the entire asset until they find a buyer for the whole property, or accept a significant discount in a distress sale. This traps capital for periods ranging from months to decades.

The transaction process itself is extraordinarily slow. Even in efficient real estate markets, a successful property sale from listing to closing typically takes 45–90 days in the United States, 2–6 months in India, and potentially longer in illiquid markets. During this period, the investor's capital is completely locked, preventing reallocation to other opportunities.

Existing blockchain tokenization platforms (RealT, Lofty.ai) attempt to solve this but fail to deliver a complete solution. They tokenize properties and allow fractional ownership, but provide no secondary market for tokens. Investors remain locked into their positions until the underlying property is liquidated — replicating the illiquidity problem in a new technical format.

4.2 Problem 2: High Entry Barriers

Purchasing even a modest apartment in a major urban centre requires upfront capital in the range of $50,000 to $500,000+. When combined with stamp duty (3–8% of property value in most jurisdictions), legal fees, mortgage processing costs, broker commissions (2–6%), and property registration charges, the effective minimum investment for retail participation is completely prohibitive for the vast majority of the global population.

High-yield investment properties — commercial buildings, luxury residential, industrial warehousing — are typically accessible only to ultra-high-net-worth individuals or large institutional investment funds. The asymmetry is stark: the asset class with the highest historical risk-adjusted returns is reserved for those who need it least.

4.3 Problem 3: Opaque and Inefficient Pricing

Real estate prices are determined through private, bilateral negotiations between buyer and seller, typically mediated by brokers acting with conflicting incentives. Comparable transaction data ("comps") used to value properties is often 3–6 months old and may not reflect current market conditions. There is no continuous, transparent, publicly available price feed for any individual property. This opacity creates significant information asymmetry between institutional investors (who can afford expensive market research) and retail investors (who cannot). It also makes real estate systematically susceptible to broker-driven price manipulation and valuation fraud.

4.4 Problem 4: The Technology Gap

While blockchain technology offers theoretical solutions to all three problems, existing blockchain tokenization platforms have failed to deliver a complete solution due to the following specific technology gaps:

- No secondary market: Tokenized assets cannot be traded after initial purchase.
- No algorithmic pricing: Token prices are set administratively, not by market demand.
- No SaaS scalability: Platforms serve a single client and cannot be white-labeled for B2B deployment.
- No production-grade reliability: Most platforms are prototypes, not live production systems serving real users with real capital.

4.5 Problem 5: Libertum's Solution Statement

Libertum directly addresses all identified problems through a production-deployed, multi-component blockchain ecosystem:

- **Illiquidity** → Fractional tokenization combined with a live P2P secondary market provides investors with an instant exit mechanism at any time, at a user-defined price.
- **High Entry Barriers** → ERC-20 fractional tokens allow participation with any investment amount, democratizing access to premium real estate.
- **Pricing Opacity** → The Bonding Curve DEX provides real-time, transparent, supply-driven price discovery governed entirely by a mathematical function, with no human intermediaries.
- **Technology Gap** → Libertum is a production-deployed platform, live on the Base blockchain, with 8+ microservices serving real users and real transactions.

4.6 Problem 6: Lack of Trust and Settlement Security

Traditional peer-to-peer property transactions rely heavily on intermediaries such as brokers, lawyers, and financial institutions to complete settlement. While these intermediaries provide oversight, they also increase transaction costs and create delays in ownership transfer. Buyers often face uncertainty regarding whether the seller will complete the transfer after payment, while sellers remain exposed to delayed payments or failed transactions. This dependency on intermediaries makes the transaction process slower and less efficient.

In blockchain-based real estate platforms, settlement becomes even more sensitive because digital assets can be transferred instantly while payment confirmation may occur separately. Without an automated mechanism to coordinate both actions, one party may fulfil their obligation while the other fails to complete theirs. This creates settlement risk and reduces investor confidence. Libertum addresses this problem through smart contract escrow, where tokens are securely locked when an order is created and are released only after payment confirmation is completed. This ensures transaction transparency, reduces manual intervention, and provides a more secure trading experience for investors.

4.7 Problem 7: Regulatory and Compliance Complexity

Real estate investment is closely regulated and requires strict verification of investor identity, ownership records, and transaction approval. Traditional systems manage compliance through paperwork and manual review, which is often slow and difficult to audit. Blockchain networks, on the other hand, are designed to be permissionless, allowing any wallet address to interact with the network. This creates a major challenge for tokenized real estate platforms because financial assets cannot be transferred freely without meeting legal and regulatory requirements. Many tokenization platforms treat compliance as a separate external process rather than integrating it into the transaction lifecycle.

4.8 Problem 8: Delayed Blockchain Event Visibility

Blockchain transactions are asynchronous and do not confirm immediately after submission. Once a transaction is broadcast to the network, it may remain pending until it is included in a confirmed block. During this period, backend services may not know whether the transaction has succeeded, failed, or is still waiting for confirmation. This creates delays in updating balances, order status, and investor notifications. Without a reliable event monitoring system, users may experience outdated information or repeated transaction processing. These issues reduce platform trust and negatively affect the user experienc

5. SCOPE OF THE PROJECT

5.1 Included in Scope

The Libertum project scope encompasses the following components, all of which are production-deployed and operational:

- **Real Estate Asset Tokenization:** ERC-20 and ERC-1155 token deployment on the Base blockchain representing fractional property ownership. Full offering lifecycle management from Draft to Active to Fully Funded to Closed.
- **P2P Order Matching Engine:** Complete peer-to-peer trading infrastructure including listing creation, order placement, payment confirmation, token release, and automated expiry handling. Includes MongoDB aggregation pipelines, Kafka event propagation, and WebSocket real-time notifications.
- **Bonding Curve DEX:** Algorithmic token pricing using a linear bonding curve model. Includes 24-hour volume computation, price chart time-series data, Etherscan-enriched holder analytics, and portfolio cumulative value tracking.
- **KYC/KYB Compliance Layer:** Third-party identity verification (SumSub integration) for individual investors (KYC) and institutional clients (KYB). All trading and investment activities are gated by verification status.
- **Admin Panel Backend:** Internal management API for offering approvals, user oversight, P2P order visibility, and platform analytics.
- **Transfer Agent Module:** Regulatory-compliant token transfer authorization, investor cap table maintenance, and full transfer history audit trail.
- **Blockchain Event Tracking Service:** Real-time on-chain log monitoring with block-number-based deduplication in PostgreSQL. Configurable batch polling with 12-block confirmation buffer.
- **Email Notification Service:** EJS-rendered transactional HTML emails covering all investor lifecycle events with per-tenant branding support.
- **Multi-Tenant SaaS Backend:** 4-step white-label product onboarding with automated Cloudflare domain provisioning and Stripe billing integration.
- **Infrastructure:** Docker containerization, GitHub Actions CI/CD with manual approval gates, Nginx reverse proxy routing, PM2 process management, and AWS/Ubuntu VPS production hosting.
- **Inter-Service Messaging:** Apache Kafka for async event propagation between Marketplace, Notification, Admin, and Socket services.

• **WebSocket Real-Time Notifications:** Browser push notifications for order status updates, delivered via Kafka-backed WebSocket relay.

5.2 Excluded from Scope

• Fiat on-ramp/off-ramp: No direct payment gateway integration. Users must bring their own stablecoins (USDC) for settlement.

• Mobile application: The platform is web-only. No native iOS or Android applications.

• Cross-chain asset bridging: Assets are deployed on Base blockchain; cross-chain transfer between Base and other networks is not supported.

• Legal or regulatory licensing obligations: The platform provides technical infrastructure; legal and regulatory licensing is the responsibility of the platform operator.

• On-chain governance (DAO): Token-holder voting and decentralized governance mechanisms are planned for a future phase.

5.3 Target Users

Table 3: Target User Segments for the Libertum Platform

User Type	Description
Retail Investors	Individuals seeking fractional real estate exposure with small capital amounts, using P2P trading and Bonding DEX.
Real Estate Developers / Issuers	Companies tokenizing properties for fundraising, managing investor relations, and tracking token distributions via the Admin Panel.
B2B SaaS Clients	Real estate platforms wanting a white-label tokenization marketplace under their own brand and domain.
Transfer Agents	Institutional entities managing security token compliance, cap table records, and regulatory transfer authorizations.
Platform Administrators	Internal operators overseeing offerings, user KYC statuses, P2P activity, and platform-wide analytics.

6. TOOLS AND TECHNOLOGIES

The Libertum platform leverages a carefully selected, production-proven technology stack across all layers of its architecture. Each tool was selected based on specific technical criteria relevant to high-reliability, high-throughput financial systems.

6.1 Programming Languages

Table 4: Programming Languages Used

Tool / Technology	Purpose
TypeScript	Primary language for all 8+ backend microservices. Provides static typing and compile-time type safety, catching entire classes of runtime errors before deployment.
JavaScript (Node.js)	Runtime environment for all server-side backend logic. Node.js's event-driven, non-blocking I/O model is ideal for handling concurrent blockchain event monitoring and API requests.
Solidity	Smart contract programming language for token minting logic, bonding curve pricing contracts, and on-chain escrow settlement mechanisms deployed on the Base blockchain.
SQL	Relational query language for PostgreSQL databases — used in the blockchain event tracking service for block number deduplication and audit logging.

6.2 Frameworks and Libraries

Table 5: Frameworks and Libraries Used

Tool / Technology	Purpose
Express.js	Lightweight REST framework used in the Bonding DEX Service, Blockchain Event Service, Notification Service, and Admin backend for their simpler routing requirements.
Ethers.js	Primary blockchain interaction library. Reads on-chain events, calls smart contract functions, manages wallet connections, and handles ERC-20 token balance queries.

Web3.js	Used in the Event Service for ABI-based contract instantiation, past event fetching across block ranges, and current block number reads.
Viem	Lightweight, high-performance EVM interaction library used in specific modules requiring optimized contract calls.
Hardhat	Smart contract development framework for Solidity compilation, local blockchain simulation, unit testing, and deployment scripting.
EJS (Embedded JS)	Templating engine for dynamic HTML email rendering in the Notification microservice, covering all investor lifecycle events.
Joi	Schema-based API request validation used across all REST endpoints for robust input sanitization and detailed error feedback.
Bull / BullMQ	Background job queuing library for cron-based blockchain event polling, asynchronous notification dispatch, and configurable retry mechanisms.
Nodemailer	Email delivery library for sending all transactional HTML emails via SMTP in the Notification service.
Big.js	High-precision decimal arithmetic library used in the Bonding DEX and P2P service to prevent floating-point precision errors (e.g., the 179.99... precision bug) in all financial calculations.
kafkajs	Node.js Kafka client for async inter-service messaging between P2P, Admin, Notification, and Socket services.

6.3 Databases

Table 6: Databases Used

Tool / Technology	Purpose
PostgreSQL	Primary relational database for data requiring ACID compliance: blockchain event block tracking, production event logs, and structured audit records.

MongoDB	Document-based NoSQL database for schema-flexible data: P2P listings and orders, user profiles, offering metadata, bonding DEX records, and notification logs.
Redis	In-memory data store used as the message broker backend for Bull/BullMQ job queues, session caching, and distributed lock management for concurrent operations.

6.4 Blockchain and Web3 Technologies

Table 7: Blockchain and Web3 Technologies

Tool / Technology	Purpose
Base (EVM Layer 2)	Primary production blockchain. EVM-compatible Layer 2 built on Coinbase's OP Stack. Sub-cent transaction fees, Ethereum security guarantees, and growing institutional adoption.
Ethereum Sepolia	Testnet for smart contract validation and integration testing before Base mainnet deployment.
ERC-20 Standard	Token standard for fungible fractional property tokens (each token = one ownership unit fraction).
ERC-1155 Standard	Multi-token standard enabling a single contract to represent multiple property assets.
Gnosis Safe (Multi-sig)	Multi-signature wallet for secure admin treasury and platform fee management operations.
Etherscan API	Used in the Bonding DEX service to fetch real token holder counts per contract address for enriched DEX discovery listings.
BondingToken ABI	Custom Solidity contract ABI for interacting with the on-chain bonding curve contract (buy, sell, price query functions).

6.5 DevOps and Infrastructure

Table 8: DevOps and Infrastructure Tools

Tool / Technology	Purpose
Docker	Containerizes each microservice for consistent, portable deployment across development and production environments.
Docker Compose	Orchestrates the full multi-container local development environment (all 8 services + databases + Kafka).
GitHub Actions	Automated CI/CD pipelines with build, test, and deploy stages. Manual approval gates prevent accidental releases to production.
Nginx	Reverse proxy routing external HTTPS traffic to individual microservice containers by URL path or subdomain rules.
AWS / Ubuntu VPS	Cloud production hosting infrastructure for all deployed microservices.
PM2	Node.js process manager for production uptime monitoring and automatic restart on service crash.
Cloudflare API	Used in the SaaS Backend to programmatically whitelist new tenant domain routes via the Cloudflare Workers Routes API when a new white-label product is activated.

6.6 Security and Compliance Tools

Table 9: Security and Compliance Tools

Tool / Technology	Purpose
SumSub KYC/KYB API	Third-party identity verification for individual investors (KYC) and institutional businesses (KYB). Blocks unverified users from trading or investing.
JWT (JSON Web Tokens)	Stateless authentication across all protected API routes and service-to-service calls.

bcrypt	Password hashing for secure user credential storage.
Helmet.js	HTTP security headers middleware protecting APIs against XSS, clickjacking, MIME-type sniffing, and other common web vulnerabilities.
Sentry	Production error tracking and alerting, integrated into the Bonding DEX and Blockchain Event services for real-time error reporting.

7. METHODOLOGY

The Libertum project followed a structured, iterative development methodology combining elements of Agile sprint planning with production-focused delivery milestones. The overall development lifecycle proceeded through six distinct phases.

7.1 Planning

The planning phase began with a comprehensive audit of existing real estate tokenization platforms to identify their limitations. The analysis confirmed three critical gaps: (i) absence of secondary trading markets, (ii) no algorithmic pricing mechanisms, and (iii) no multi-tenant SaaS scalability.

Based on this analysis, the team defined a microservices architecture comprising 8+ isolated services, each with distinct responsibilities and dedicated databases. The strict no-shared-database principle was established from the outset: all cross-service data reads must occur through REST API calls, never by directly querying another service's database. This architectural decision ensures that services remain independently deployable and scalable.

The Base blockchain (EVM Layer 2) was selected as the primary deployment target based on three criteria: (i) transaction fees below $0.01 per transaction, enabling viable retail usage; (ii) full Ethereum Virtual Machine (EVM) compatibility, allowing use of the established Solidity/Hardhat toolchain; and (iii) institutional backing from Coinbase providing long-term network stability. After planning was finalized, the project moved into the system design phase where the technical architecture, service boundaries, data flow, and blockchain interaction model were formally defined. Since Libertum required real-time communication, blockchain synchronization, investor compliance, and multi-tenant deployment within a single ecosystem, a distributed microservices architecture was adopted instead of a monolithic backend. Each major business capability was separated into an independent service including

Marketplace, Admin, Notification, Socket, Blockchain Event Tracker, Tenant Management, Compliance, and User Management. Every service maintained its own database and internal business logic. This separation improved scalability, fault isolation, and deployment flexibility. If one service experienced failure or required an update, other services could continue operating without interruption.

The communication model between services was carefully designed using a combination of REST APIs and Apache Kafka. REST APIs were used for synchronous request-response operations where immediate validation or data retrieval was required. Kafka was implemented for asynchronous event-driven workflows such as order placement, email triggering, notification delivery, and blockchain event propagation. This hybrid communication model ensured both real-time responsiveness and high system reliability.

Database architecture was designed according to the nature of each service. PostgreSQL was selected for relational and transactional consistency, while MongoDB was used for flexible document storage and aggregation-heavy marketplace operations. This database separation improved performance and ensured that each service used the most suitable storage technology.

At the blockchain layer, smart contracts were designed using Solidity and developed through the Hardhat framework. Contracts were structured to manage token issuance, fractional ownership, token transfers, and exchange interactions. The Base blockchain network was integrated using RPC providers, allowing secure deployment and interaction with smart contracts. Wallet integration was designed to allow users to sign blockchain transactions directly while maintaining security and ownership control.

A real-time notification architecture was also included during design. WebSocket channels were connected with Kafka event consumers to instantly push platform updates to investors whenever orders changed status, payments were confirmed, or compliance approvals were completed. This architecture ensured minimal delay between backend processing and frontend visibility.

The system design phase also included API contract definition, request/response validation planning, environment variable structuring, and service deployment workflows. By the end of this phase, a complete technical blueprint was established, reducing implementation risk and improving development efficiency.

The full investor journey was mapped end-to-end to ensure no gaps existed in the user experience flow:

1. User registers on the platform.
2. KYC/KYB identity verification completed.
3. User browses active property offerings on the marketplace.
4. User invests in the primary market (token minting).
5. User accesses P2P secondary market (to trade existing tokens).
6. User accesses Bonding DEX (to buy/sell via algorithmic pricing).
7. On-chain tokens received in user's wallet.
8. Blockchain Event Service tracks all on-chain activity.
9. Email notifications dispatched at each lifecycle milestone.

7.2 Research

The research phase focused on five technical domains critical to the platform's design:

- **Bonding Curve Mathematics:** Studied Uniswap V2 constant-product AMM (x * y = k), Bancor Protocol linear bonding curves, and academic literature on mechanism design for token pricing. Selected the linear model for real estate tokens due to its intuitive price appreciation characteristics.
- **Security Token Standards:** Reviewed ERC-1404 (Simple Restricted Token Standard) and ERC-3643 (T-REX Protocol) for their approaches to on-chain KYC enforcement and transfer restriction logic.
- **P2P Protocol Design:** Analysed OpenSea's escrow-based settlement protocol to understand trustless asset custody patterns applicable to security tokens.
- **NestJS Architecture:** Studied NestJS dependency injection patterns, module isolation strategies, and interceptor/guard chains to design modular, testable service components.
- **Kafka vs. REST for Inter-Service Communication:** Evaluated Apache Kafka against REST for microservice communication. Chose Kafka for async, decoupled messaging because it provides durable message persistence, configurable consumer group replay, and natural fanout (one publisher, many subscribers).

A major area of research involved decentralized token pricing and market behavior. Different automated pricing models were studied to understand how token values could be managed transparently without centralized intervention. This included analysis of decentralized exchange mechanisms, bonding curve models, and token economics literature. After

evaluating these approaches, the linear bonding curve model was considered the most suitable because it provides predictable and transparent price movement, which closely aligns with real estate investment where value typically appreciates gradually and investors prefer stable pricing visibility.

The project also required research into blockchain token standards specifically designed for regulated assets. Existing Ethereum-based standards for security tokens were reviewed to understand how compliance restrictions and controlled transfers could be handled on-chain. This analysis helped define the token ownership and transfer structure used in Libertum while maintaining compatibility with the Ethereum Virtual Machine and allowing future scalability.

Another important research area involved peer-to-peer marketplace architecture. Existing blockchain marketplaces were studied to understand secure asset exchange, escrow-based settlement, and ownership transfer mechanisms between buyers and sellers. These findings were important in designing Libertum's secondary market because token transfers needed to remain secure, transparent, and traceable while reducing operational friction between investors.

Backend architectural research focused on NestJS and distributed microservice design. The framework was studied in depth to understand dependency injection, modular service structure, controller-based request handling, middleware usage, and scalable project organization. This research supported the decision to separate the platform into isolated services, improving maintainability, independent deployment, and long-term scalability.

A comparative study was also conducted on inter-service communication patterns. REST-based communication was reviewed for direct request-response operations, while Apache Kafka was studied for asynchronous messaging and event-driven workflows. Kafka was selected for background operations because it provides reliable message persistence, supports multiple subscribers for the same event, and reduces direct dependency between services. This architecture was particularly useful for marketplace activity, notifications, and blockchain event propagation. Additional research was performed on blockchain infrastructure and deployment environments. Multiple EVM-compatible networks were evaluated based on gas fees, transaction speed, ecosystem maturity, and long-term reliability. Base blockchain was selected because it offers low transaction cost, Ethereum compatibility, and a stable network environment suitable for production deployment.

Database technologies were also reviewed to determine the most effective storage model. Relational and document-based database systems were analyzed based on performance and data structure requirements. This research led to adoption of a hybrid storage approach, where

structured data and event tracking could be maintained efficiently while still supporting flexible marketplace operations.

Real-time communication methods were also examined. Approaches such as API polling and WebSocket-based communication were compared to determine the most responsive user experience. WebSockets were selected because they provide instant event delivery and improve real-time visibility for investor actions such as order placement and transaction status updates.

Overall, the research phase provided the technical and architectural foundation for Libertum. It ensured that blockchain integration, pricing mechanisms, backend architecture, compliance workflows, and communication patterns were all carefully evaluated before development began. This phase reduced implementation uncertainty, supported informed decision-making, and established a clear technical direction for building a secure, scalable, and production-ready platform.

7.3 Design

The design phase translated research findings into concrete architectural and data model specifications.

7.3.1 Service Boundary Design

Each microservice was assigned exclusive ownership of its data store. The Marketplace Service owns the MongoDB collections for users, offerings, orders, and P2P listings/orders. The Bonding DEX Service owns its transactions collection. The Blockchain Event Service owns the PostgreSQL blocks tracking table. Cross-service reads happen through authenticated REST API calls only. Each microservice was assigned exclusive ownership of its data store to maintain separation of concerns and reduce interdependency. The Marketplace Service owns the MongoDB collections for users, offerings, investments, orders, and P2P listings. The Bonding DEX Service manages its own transaction history, pricing data, and trading records. The Blockchain Event Service maintains PostgreSQL tables for processed block checkpoints and event tracking. The Notification Service manages email templates and delivery logs, while the Admin Service handles moderation records and internal platform controls.

Cross-service communication was intentionally restricted to authenticated REST API calls and Kafka-based event messaging. No service was permitted to directly query another service’s database. This design ensured strong encapsulation and prevented accidental dependency between internal data structures. It also simplified deployment because each service could be updated independently without impacting database access patterns in other modules. The boundary design also improved fault tolerance. If one service became

temporarily unavailable, other services could continue functioning independently and resume synchronization once communication was restored. This structure was particularly important for blockchain event handling and marketplace workflows, where uninterrupted processing is critical. The result was a modular and scalable architecture capable of supporting production workloads while remaining easy to maintain and extend

7.3.2 P2P Order Lifecycle State Machine Design

The P2P order lifecycle was modelled as a deterministic finite state machine with seven states:

PENDING → ACTIVE → PAYMENT_PENDING → PAYMENT_CONFIRMED → TOKENS_RELEASED → COMPLETED

Each state transition has a defined trigger, side effects (Kafka events, emails, WebSocket messages), and failure modes (CANCELLED, EXPIRED with quantity restoration). This formal state machine design prevents race conditions and ensures consistent system state across distributed service restarts. For example, an order begins in **PENDING** when created by a seller. Once it becomes visible in the marketplace, it transitions to **ACTIVE**. When a buyer places an order, the state changes to **PAYMENT_PENDING**, followed by **PAYMENT_CONFIRMED** after settlement validation. Once blockchain transfer conditions are verified, the system moves to **TOKENS_RELEASED**, and finally to **COMPLETED** when ownership is updated successfully.

Each transition triggers internal actions including Kafka event publication, email notification delivery, database updates, and WebSocket-based real-time notifications. Exception handling was also defined in detail. If payment is not completed within the configured time window, the order expires automatically and token quantity is restored. If cancellation occurs, state rollback logic ensures investor balances remain accurate.

This formal state machine design reduced race conditions and ensured consistent behavior across distributed services. Because all transitions are deterministic, system recovery after restart remains predictable, and partial transactions can be resumed safely without corrupting order state

7.3.3 Bonding Curve Price Formula Design

The linear bonding curve price formula was defined as:

Price(supply) = BasePrice × (1 + k × CurrentSupply)

Where BasePrice is the initial token price set at launch, k is a per-offering curve steepness constant, and CurrentSupply is the current number of tokens in circulation. The constant k is calibrated per property to model expected appreciation rates. This model was selected because

it produces gradual and predictable price movement that closely matches expected appreciation behavior in real estate-backed assets. Unlike highly volatile AMM-based pricing models, the linear curve allows investors to understand how price changes as demand increases.

The design also included dynamic buy and sell calculations, cumulative portfolio value tracking, and transaction fee estimation. Each property offering can define a different curve constant **k** depending on investment strategy, expected growth, or liquidity requirements. This allows flexible pricing behavior per asset while maintaining a consistent mathematical structure across the platform.

The system was also designed to calculate real-time token prices whenever supply changes due to purchase or sale. Historical price points are stored and exposed to investors through charts and analytics dashboards. This provides pricing transparency and enables informed trading decisions.

By defining the bonding curve mathematically during the design phase, implementation became predictable and testing could be performed using deterministic output values.

7.3.4 Event Deduplication Design

The Blockchain Event Service was designed with a critical deduplication mechanism. A PostgreSQL table stores the last successfully processed block number per (chain, contract_address) combination. On each polling cycle, the service reads this checkpoint and processes only new blocks. On crash/restart, it resumes from the last checkpoint rather than from block zero, guaranteeing exactly-once event processing semantics. This design prevents duplicate event execution even if the polling cycle repeats or temporary failures occur. In case of server crash or restart, the service resumes from the stored checkpoint rather than scanning from the beginning of the blockchain. This significantly improves performance and prevents repeated transaction processing.

A confirmation buffer was also included in the design. Rather than processing the latest mined block immediately, the service waits for a defined number of confirmations before marking events as valid. This reduces the risk of temporary blockchain reorganization affecting system accuracy.

Each processed event is validated and mapped to internal business actions such as ownership updates, order completion, or investor notifications. Duplicate prevention was critical because repeated execution could result in incorrect balances or duplicate notifications.

This design guarantees near exactly-once event processing semantics and ensures that on-chain activity remains synchronized with off-chain application records in a reliable and production-safe manner.

Overall, the design phase transformed research and planning decisions into a structured technical blueprint. By clearly defining service ownership, transaction state management, pricing formulas, and blockchain synchronization rules, Libertum established a scalable architecture capable of supporting real-time real estate tokenization and trading in a production environment.

7.4 Implementation

The implementation phase is detailed extensively in Section 8 (Modules / Features). Key implementation principles enforced throughout:

- All financial calculations performed using Big.js to prevent floating-point precision errors inherent in JavaScript's IEEE 754 floating-point representation.
- All MongoDB aggregation pipelines use $facet for simultaneous data and count retrieval, eliminating redundant database round-trips.
- All external API calls (Etherscan, Cloudflare, KYC provider) wrapped in try-catch blocks with Sentry error logging.
- Kafka message publishing implemented with idempotent producer configuration and retry logic.
- All REST endpoints validated through Joi schema middleware before reaching service logic.

7.5 Testing

Testing is a critical phase in blockchain-enabled distributed systems because application correctness directly affects user funds, transaction integrity, and platform trust. Unlike conventional web applications, decentralized finance and tokenized asset platforms must validate both off-chain business logic and on-chain contract interactions. A minor logic error in trade execution, state synchronization, or event handling can result in incorrect ownership records, failed settlements, or inconsistent blockchain data. Therefore, Libertum followed a structured multi-level testing approach to validate business workflows, service integration, and blockchain synchronization before deployment. The testing strategy was designed to ensure reliability across all layers of the architecture. Unit testing was used to validate isolated business logic at the service level. Integration testing verified communication between multiple microservices and blockchain components in real execution environments.

Blockchain event testing focused specifically on ensuring consistency between on-chain state and backend databases. This layered approach helped identify logic issues early, reduce production risk, and ensure stable operation under realistic platform conditions Testing was conducted at three levels:

7.5.1 Unit Testing

Unit testing was performed to validate individual service functions independently from external dependencies such as databases, blockchain providers, and messaging queues. The primary objective was to ensure that each module behaved correctly under normal and edge-case conditions before integration with other services. This level of testing is especially important in financial systems because transaction-related functions often involve multiple validation rules and precision-sensitive calculations.

For the Libertum platform, unit testing focused heavily on the P2P trading engine and bonding curve pricing modules because both directly impact token ownership and transaction execution. Mocked dependencies were used to isolate business logic and verify behavior across expected scenarios. Edge cases such as invalid order quantities, expired listings, balance validation failures, and token pricing calculations were tested repeatedly to confirm deterministic output and prevent incorrect order execution.

P2P matching logic was unit tested with comprehensive edge case coverage:

- Partial order fills when available quantity is less than the requested quantity.
- Price mismatch rejection between buyer and seller listings.
- Order expiry: verification that quantity is correctly returned to listing's availableQuantity field.
- Insufficient token balance validation using mocked web3Helper.getTokenBalance() calls.
- Bonding curve price calculations across the full supply range (0 to maximum supply) verified against hand-computed expected values.

7.5.2 Integration Testing

While unit testing validates isolated modules, integration testing verifies whether multiple services work correctly together under realistic conditions. Since Libertum is built as a distributed microservice platform, integration testing was necessary to confirm that APIs, Kafka events, database transactions, and blockchain interactions remained synchronized throughout the complete investor workflow.

A staging environment connected to the Ethereum Sepolia testnet was used to simulate real blockchain execution while avoiding production financial risk. This enabled testing of smart

contract transactions, backend workflows, and asynchronous event handling in conditions closely matching the production environment. Integration testing also validated that status transitions remained consistent across services and that no failures occurred between blockchain confirmation and backend processing.

Full P2P trade lifecycle was tested end-to-end on the Ethereum Sepolia testnet:

1. Create SELL listing → verify status transitions to ACTIVE.
2. Buyer places order → verify availableQuantity decremented atomically.
3. Payment confirmation → verify status transitions to PAYMENT_CONFIRMED.
4. Seller releases tokens → verify status transitions to TOKENS_RELEASED → COMPLETED.
5. Verify P2P_BUY and P2P_SELL records created in OrderSchema.
6. Verify Kafka events published with correct payloads.
7. Verify email notifications dispatched to both buyer and seller.

7.5.3 Blockchain Event Service Testing

Multi-block batch polling tested across ranges with deliberate deduplication scenarios: service restarted mid-batch to verify that no events were duplicated or missed on resumption. Blockchain event synchronization required dedicated testing because backend services depend on accurate on-chain event indexing. Any duplicated or missed blockchain event can create inconsistent database records, incorrect balances, or delayed UI updates. This is particularly important in real estate tokenization where ownership and settlement records must remain auditable and synchronized at all times.

The blockchain event service was tested across multiple block ranges using controlled restart scenarios to simulate temporary service interruption. The objective was to verify checkpoint recovery and ensure that event polling resumed from the correct block without duplication or omission. Batch processing was also tested to confirm stable performance across larger ranges of blockchain activity. Multi-block batch polling tested across ranges with deliberate deduplication scenarios: service restarted mid-batch to verify that no events were duplicated or missed on resumption.

This testing improved confidence in long-running blockchain synchronization reliability and reduced operational risk during production deployment.

Deployment of Libertum followed a containerized microservice architecture to ensure environment consistency, scalability, and simplified release management. Since the platform contains multiple independent services—including trading, compliance, blockchain listeners, notifications, and administrative modules—containerization ensured that each service could

run with isolated dependencies while remaining portable across development, staging, and production environments.

All microservices were packaged with dedicated Dockerfiles and environment-specific configuration variables. This made local onboarding easier for developers while maintaining consistent runtime behavior across different environments. Docker Compose was used to orchestrate the full development stack, including backend services, databases, Kafka brokers, and supporting infrastructure, allowing complete end-to-end testing from a single controlled environment.

All 8+ microservices were containerized with individual Dockerfiles and service-specific environment variable files. Docker Compose orchestrates the full local development environment.

Continuous integration and deployment were automated using GitHub Actions. The CI/CD pipeline was configured to trigger automatically on every push to the main branch. Automated workflows performed dependency installation, code quality checks, test execution, and build validation before deployment. This reduced manual intervention, improved release consistency, and ensured that production updates were deployed only after passing validation checks.

7.6 Deployment

All 8+ microservices were containerized with individual Dockerfiles and service-specific environment variable files. Docker Compose orchestrates the full local development environment.

GitHub Actions CI/CD pipeline triggers on push to the main branch:

1. Build: Docker images built for all changed services.
2. Test: Automated test suite executed.
3. Manual Approval Gate: A human reviewer must approve before production deployment.
4. Deploy: Images pushed to registry and containers updated on production VPS.
5. MongoDB schema updates and index migrations executed before service startup to ensure compatibility with new releases and maintain query performance.
6. Separate .env files maintained for development, staging, and production to securely manage API keys, blockchain RPC endpoints, database credentials, and third-party integration settings.
7. Each container exposes health check endpoints monitored during deployment to verify successful startup and ensure services are responding before traffic is routed.

8. Centralized application logs are captured for all microservices to monitor runtime activity, detect failures, and simplify debugging during production incidents.
9. Kafka and related event consumers are started automatically with deployment to ensure asynchronous event processing resumes without message loss.
10. Service containers are updated sequentially to minimize interruption and maintain platform availability during releases.
11. HTTPS certificates are configured through Nginx to secure all API traffic and route requests through domain-based or path-based endpoints.
12. Automated database backups and deployment rollback procedures are maintained to restore system stability in case of deployment failure.
13. Post-deployment smoke testing verifies key platform workflows such as login, blockchain connectivity, Kafka event publishing, and API responsiveness.
14. Containerized deployment allows individual high-traffic services such as the P2P engine and blockchain listener to scale independently based on production workload.
15. Firewall rules, restricted server access, and environment secret isolation are enforced to protect infrastructure and prevent unauthorized access to production systems.
16. Role-based access control (RBAC) is implemented across admin panels, APIs, and internal services to ensure users and operators can access only authorized resources and actions.
17. Monitoring and alerting systems are integrated using tools such as Prometheus and Grafana to track CPU usage, memory consumption, API latency, container health, and Kafka consumer performance in real time.
18. Zero-downtime deployment strategies such as rolling updates or blue-green deployments are used to release new versions without interrupting active users or ongoing blockchain and P2P transactions.

7.7 System Diagrams

7.7.1 System Architecture & Workflow Flowchart

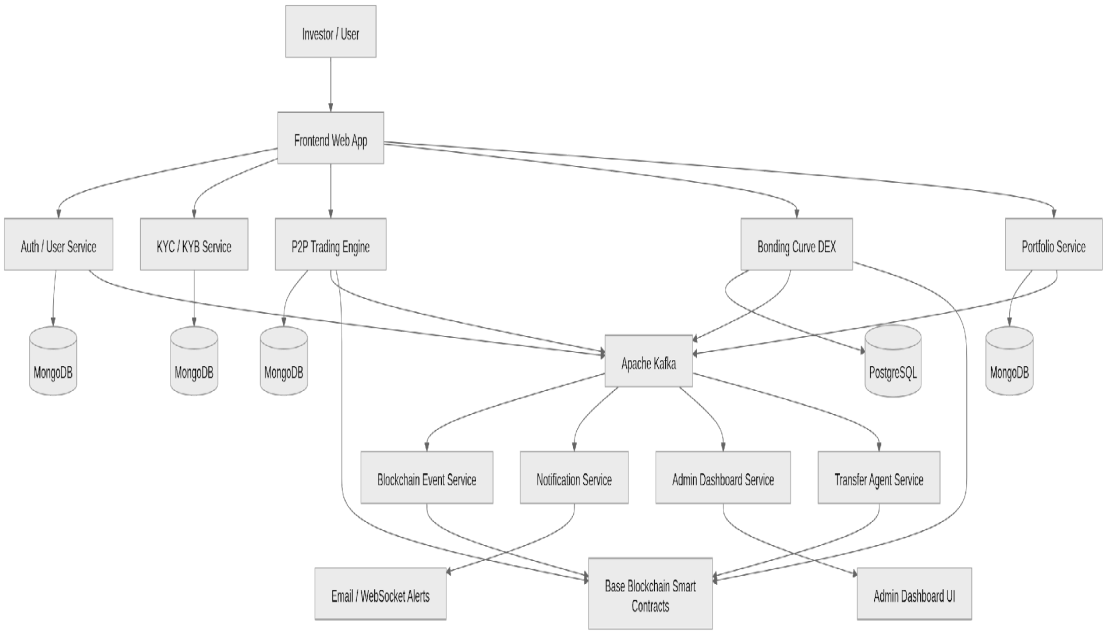

Figure 1: End-to-End Flowchart — Planning through Deployment Phases

7.7.2 P2P Work Flow Diagram

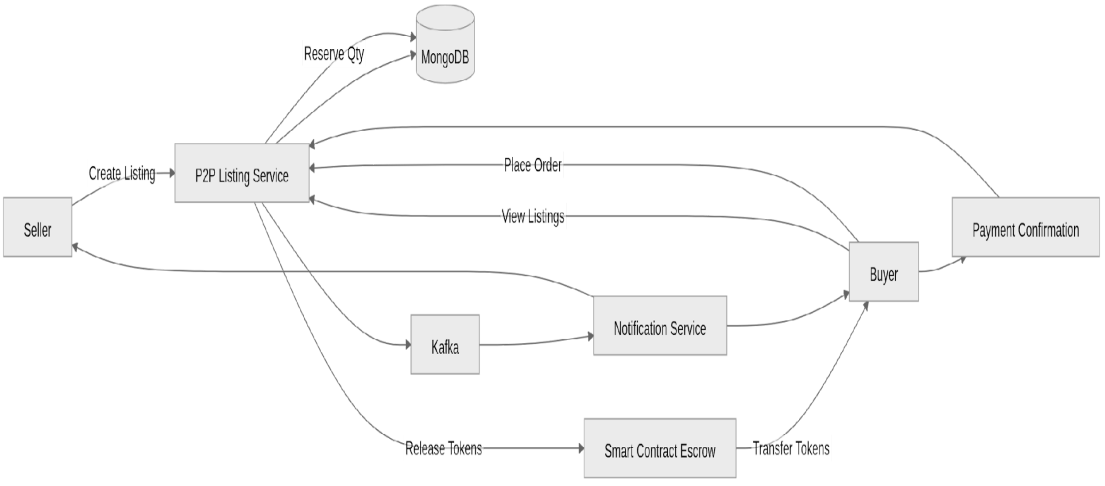

Figure 2 End-to-End Flowchart —P2P Flow

7.7.3 UML Diagram

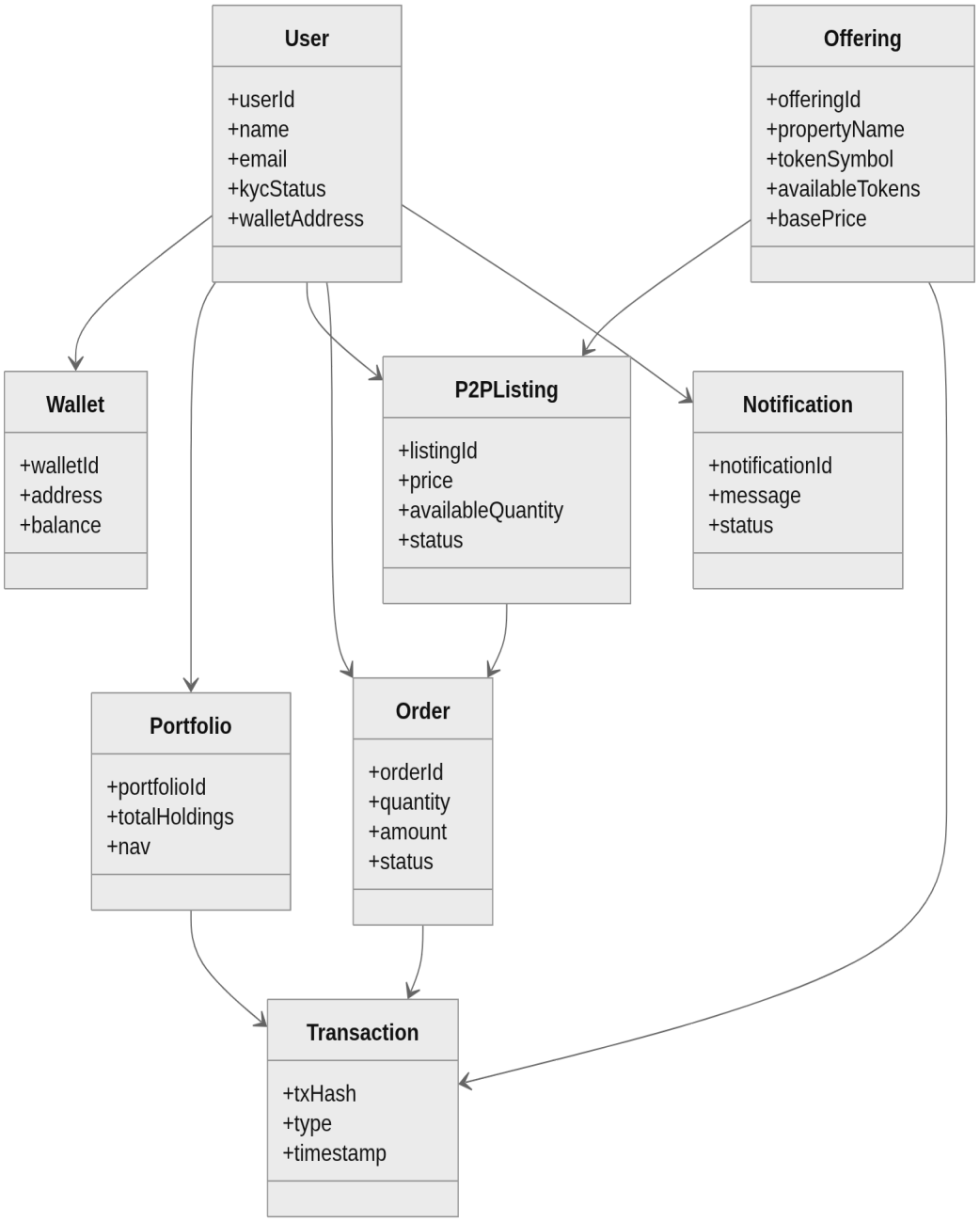

Figure 3: UML Diagram — Component Relationships & Interaction Sequences

7.7.4 Sequence Diagram

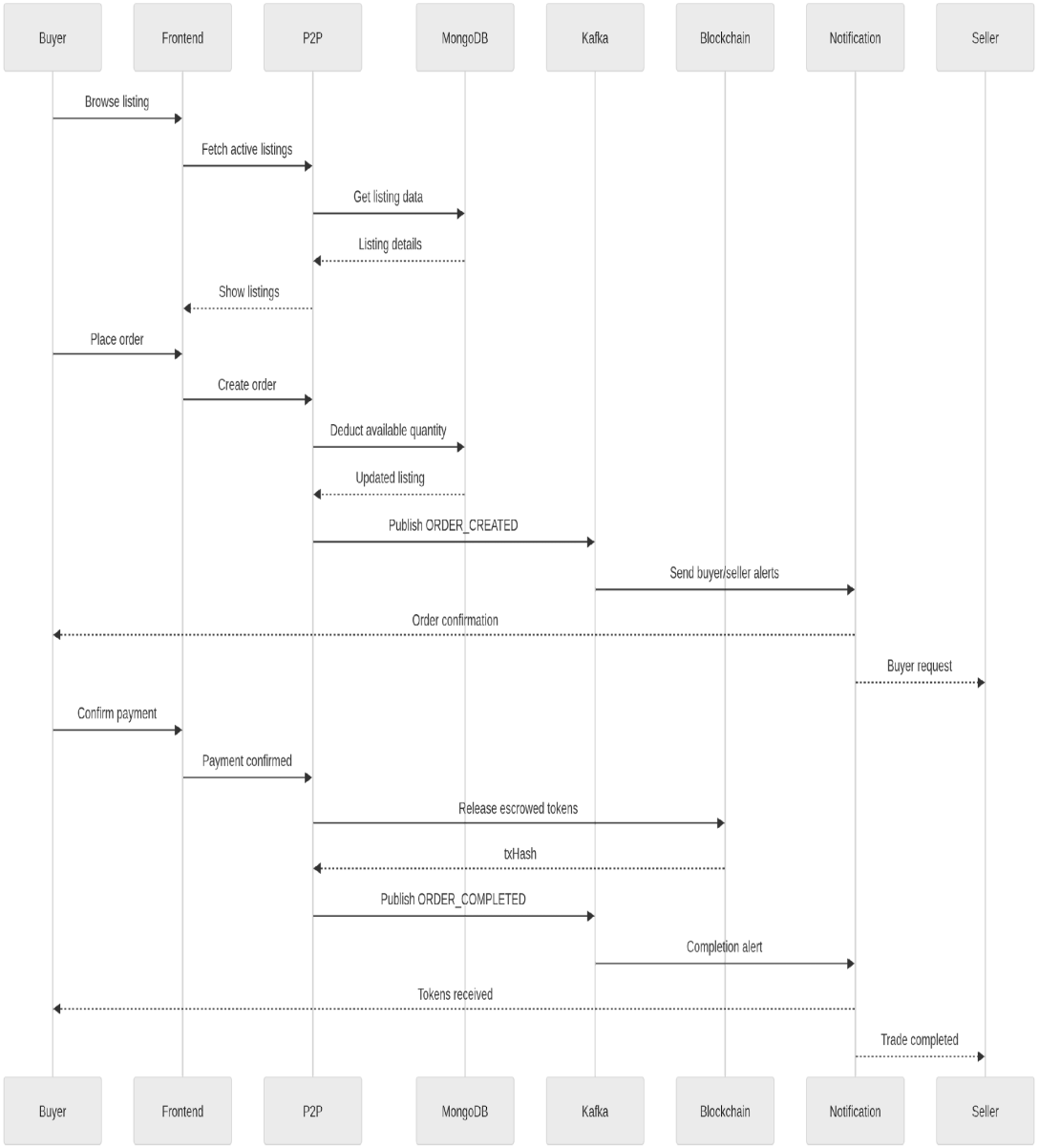

Figure 4: Sequence Diagram – P2P Investment Workflow

8. MODULES / FEATURES

The Libertum platform is composed of 8+ independently deployed microservices, each with clearly defined responsibilities and isolated data stores. This section provides a detailed technical deep-dive into each module, with particular focus on the P2P Trading Engine and Bonding Curve DEX — the author's primary areas of contribution.

8.1 P2P Trading Engine [Primary Focus]

8.1.1 Overview

The P2P engine is Libertum's secondary market mechanism. It enables any KYC-verified token holder to list their tokenized property shares for sale at a self-defined price, and any verified buyer to purchase those tokens directly — without intermediaries, without a centralized exchange, and without the need for a matching counterparty to be simultaneously online.

The engine is implemented as the P2PService class within the Backend-marketplace service (TypeScript, NestJS). It owns two primary MongoDB collections: P2PListingSchema (seller listings) and P2POrderSchema (buyer orders against specific listings).

8.1.2 P2P Order Lifecycle State Machine

The complete order lifecycle is governed by a deterministic state machine. Each state transition is atomic — the system either successfully transitions to the next state and fires all downstream effects (Kafka events, emails, WebSocket messages), or it fails entirely and the state remains unchanged. This prevents partial state corruption.

8.1.3 Key Method Implementations

A. createListing(listingData, userId) — Listing Creation

When a seller creates a listing, the system: (i) validates that the offering exists and has isActive set to true; (ii) validates that p2pEnabled is explicitly set to true on the offering (each offering has a toggle controlling P2P trading eligibility); (iii) for SELL listings, makes a real-time ERC-20 balanceOf() call via Ethers.js to verify the user's on-chain wallet balance is greater than or equal to the quantity being listed — preventing fraudulent listings by users who no longer hold the tokens; (iv) creates a P2PListingSchema document with status: PENDING; and (v) fires a Kafka P2P_LISTING_CREATED event and a WebSocket message to the user's connected browser session.

B. createOrder(tradeData, userId) — Order Placement and Lifecycle

When a buyer places an order against an active SELL listing: (i) the system validates that the requested quantity is less than or equal to listing.availableQuantity; (ii) validates price agreement between order and listing; (iii) validates that the order falls within the listing's minOrderLimit and maxOrderLimit bounds; (iv) deducts the ordered quantity from listing.availableQuantity atomically using MongoDB's $inc operator to prevent race conditions in concurrent order placements; (v) creates a P2POrderSchema document with status: PAYMENT_PENDING. The buyer then completes payment off-chain, and a subsequent API call confirms payment, transitioning the order to PAYMENT_CONFIRMED. The seller then calls the token release endpoint, transitioning the order through TOKENS_RELEASED to COMPLETED.

C. ensureP2POrdersInOrderSchema — Post-Trade Record Creation

Upon trade completion, this method creates two records in the shared OrderSchema collection — one for the buyer (type: P2P_BUY) and one for the seller (type: P2P_SELL). Critically, the buyer's amount uses the agreed p2pPrice (the traded price), while the seller's amount uses navPrice (Net Asset Value = latestNav / totalSupply) to calculate fair value for portfolio tracking purposes. Both records are immediately synced to the Admin service via Kafka so the admin dashboard reflects all P2P activity in real time. Duplicate-creation protection checks existingCount >= 2 before attempting creation, and only re-syncs to admin on subsequent calls.

D. sendP2POrderEmails — Concurrent Notification Dispatch

This method orchestrates all notifications for a P2P order status change: (i) fetches buyer and seller user details and offering name in PARALLEL using Promise.all, eliminating sequential database round-trips; (ii) resolves the correct domain prefix from the order or offering for multi-tenant email branding; (iii) dispatches EJS-rendered HTML emails to BOTH buyer and seller simultaneously using Promise.all([buyerEmail, sellerEmail]) — individual .catch() handlers on each email prevent one failure from blocking the other; (iv) publishes Kafka real-time in-app notification events; and (v) publishes Kafka WebSocket payloads so the user's browser receives instant order status updates without polling.

E. getListings — Marketplace Discovery

The listing discovery endpoint implements a multi-stage MongoDB aggregation pipeline: $match for active listings excluding the requesting user's own listings ($ne operator); $lookup joins against the users and offerings collections; and $project to return enriched listing data including seller profile, offering details, token metadata, and asset type. The endpoint supports advanced filtering by listingType (BUY/SELL), paymentMethod, minPrice,

maxPrice, text search (regex on offeringName), pagination with configurable page sizes, and a CSV export mode for data download.

F. getUserTokenBalance — On-Chain Balance Calculation

Rather than querying the blockchain for each balance check (expensive and slow), this method implements an off-chain balance reconstruction: a MongoDB aggregation over all OrderSchema records for a given user and offering computes net token balance as the sum of positive flows (MINTED, TRANSFER_TO orders) minus negative flows (TRANSFER_FROM, BURN orders). This provides a consistent, fast, database-backed balance without blockchain round-trips.

G. Expiry Handling — Automatic Order Cleanup

Orders have a configurable expiry window (measured in hours from PAYMENT_PENDING creation). A BullMQ cron job runs periodically to identify all PAYMENT_PENDING orders past their expiry time. For each expired order: (i) order status transitions to EXPIRED; (ii) the ordered quantity is returned to listing.availableQuantity using MongoDB's $inc operator; and (iii) expiry notification emails are dispatched to both buyer and seller explaining the order expiration.

8.2 Bonding Curve DEX [Primary Focus]

8.2.1 Overview and Price Model

The Bonding Curve DEX replaces a traditional order book with an algorithmic pricing contract. Token prices are calculated purely based on current circulating supply, providing a guarantee of continuous liquidity: any user can always buy or sell at the current mathematically-determined price, backed by an on-chain reserve.

The linear bonding curve price formula implemented is:

Price(supply) = BasePrice × (1 + k × CurrentSupply)

The following table illustrates the price progression for an example token with BasePrice = $1.00 and k = 0.0001:

Table 10: Bonding Curve Price Progression Example

Tokens in Circulation (Supply)	Token Price (USD)
0	$1.00
1,000	$1.10
5,000	$1.50
10,000	$2.00

This model rewards early investors with lower entry prices and creates natural scarcity-based price appreciation as market adoption grows — directly mirroring the expected behaviour of real-world property value appreciation.

8.2.2 Key Method Implementations

A. createBondingdex — Project Upsert

Creates or updates a bonding DEX project using MongoDB's findOneAndUpdate with upsert: true semantics. This idempotent operation allows the same project to be updated without risk of duplicate document creation. Each project document stores the bondingAddress (smart contract address), tokenAddress, stakingAddress, fundAddress, and a rich projectDetails object including bondingPrice, latestNav, navLaunchPrice, tokenSupply, projectedYield, and ROI calculations.

B. getBondingdex — Listing with Volume Analytics

A full MongoDB aggregation pipeline with multi-field text search (regex across title, offeringName, assetName, tokenTicker), status filtering (hiding internal workflow states), and $lookup join to the transactions collection for each listing. Per-listing computed fields include: 24hVolume (sum of amount × bondingPrice for transactions in the last 24 hours) and 24hChange (percentage change = ((latestNav - navLaunchPrice) / navLaunchPrice) × 100). Results are rounded to two decimal places using MongoDB's $round operator. Uses $facet to compute data and total count in a single aggregation pass.

C. getBondingdexWithHolderCounts — Etherscan-Enriched Data

After retrieving base listing data via getBondingdex(), this method enriches each row with live totalHolders count from the Etherscan Pro API. To avoid rate limiting, results are processed in batches of 5 using a chunk() utility, with Promise.all() parallelizing requests within each batch. Failed Etherscan calls fall back to the existing row.totalHolders value, ensuring the listing always has a holder count even during API outages.

D. getTokenChartData — Time-Series Price Chart

Supports historical period filters: '1d', '7d', '1m', '1y', 'all'. First aggregates all transactions before the selected period start to compute an initialCumulativeValue (portfolio value prior to the period). Then processes period transactions chronologically, computing cumulativeValue at each point using Big.js for 18-decimal precision arithmetic: BuyTokens events add to cumulative value; SellTokens events subtract. Output is a time-series chartData array suitable for TradingView-style candlestick or line charts.

E. getTokenHolderData — Net Holdings Calculation

Fetches all transactions for a token and builds an in-memory holdingsMap (walletAddress → netQuantity). BuyTokens transactions increment the balance; SellTokens transactions decrement. Addresses with near-zero balances (< 0.000001 epsilon threshold) are filtered out to handle floating-point edge cases. Returns a paginated, sorted unique holder list with net token quantity per address.

F. getUniqueTokenHolderCount — Unique Holder Count

A targeted MongoDB aggregation: matches all transactions for a token, groups by userAddress ($group: { _id: '$userAddress' }), then $count to obtain the total number of unique wallet addresses that have ever interacted with the token. This provides an accurate on-platform holder count independent of Etherscan.

8.3 Marketplace Microservice

Framework: NestJS with Mongoose (MongoDB). The central service managing offering lifecycle, user authentication, primary market orders, and hosting the P2P engine.

8.3.1 Offerings Module

Manages the complete offering lifecycle from Draft through Active to Fully Funded and Closed. Stores all offering metadata including property details, token contract address, NAV (Net Asset Value), total token supply, token decimals, asset type (Equity or Debt), and the p2pEnabled flag that controls per-offering P2P trading eligibility. The fetchOfferingDetails() method is called by the P2P service to validate all these fields before any P2P listing creation.

8.3.2 User Authentication Module

Implements JWT-based stateless authentication with bcrypt password hashing. User records store KYC status as a field that gates access to all trading, listing, and investment functionalities. Wallet addresses are stored in the kycDetails.wallets[] array, allowing a single user to manage multiple blockchain wallets.

8.3.3 Kafka Integration

The kafkaService component within the Marketplace service provides three publishing methods: sendMessageToAdmin() publishes order documents to the Admin kafka topic for real-time admin dashboard updates; sendRealtimeNotification() publishes notification events (title, message, payload) to the Notification kafka topic; sendMessageToSocket() publishes WebSocket payloads to the Socket kafka topic, consumed by the real-time Socket service for browser push delivery.

8.4 Blockchain Event Service

Framework: Express.js with Web3.js and PostgreSQL. This service is the synchronization engine of the entire platform — it runs autonomously without a user interface, continuously monitoring the Base blockchain for smart contract events and translating them into structured business data.

8.4.1 Block Polling Architecture

The fetch() method implements a precise block window management strategy: it reads the current blockchain block number, applies a 12-block confirmation buffer (safeBlock = currentBlock - 12) to prevent processing blocks that may still be subject to chain reorganization, reads the lastFetchedBlock checkpoint from PostgreSQL, and processes events in a batch window from lastFetchedBlock to min(lastFetchedBlock + EVENT_BATCH_SIZE, safeBlock). After successful processing, it updates the PostgreSQL checkpoint to toBlock + 1, ensuring the next batch starts exactly where this one ended.

8.4.2 Deduplication Strategy

The deduplication mechanism relies on PostgreSQL's INSERT ... ON CONFLICT DO UPDATE (upsert) semantics on the (chain, contract_address) composite key. This guarantees that the block checkpoint is always consistent — even if the service crashes and restarts mid-batch, the checkpoint is only updated after successful event processing, so no events are skipped or processed twice.

8.5 Notification Microservice

Framework: Express.js with EJS and Nodemailer. Receives notification triggers from other services via Kafka or direct REST API calls and dispatches EJS-rendered HTML transactional emails.

The service maintains EJS templates for every key investor lifecycle event: token minting confirmation, P2P order creation, payment confirmation, token release (trade complete), order cancellation with reason, order expiration, KYC approval, and KYC rejection. Each template

receives dynamic data including offeringName, quantity, totalAmount, transactionId, paymentMethod, and user name.

Multi-tenant email branding is implemented by resolving a domain prefix from the order's createdDomain field, which is then used to construct tenant-specific loginUrl values for each outgoing email — ensuring white-label clients receive fully branded communications.

8.6 SaaS Backend (Multi-Tenant)

Framework: NestJS with Stripe and Cloudflare API integration. Manages white-label B2B marketplace creation with a 4-step product onboarding workflow:

1. createProductBasic() — Validates no in-progress product exists; creates product with status: 'in-progress'.
2. Server Details — Validates uniqueness of namespace, productName, and domainPrefix globally; validates that the provided feeAddress wallet is not owned by another user (cross-service check via Marketplace REST API).
3. Platform Details — Updates branding: title, description, favicon, logo, banner, theme colors (primary, secondary, border), and store details.
4. Plan Assignment — Assigns a subscription plan, transitions status to 'pending' (triggers admin review), and notifies all admin users.

On final activation, the whitelistDomainRoute() method makes a POST request to the Cloudflare Workers Routes API to whitelist the tenant's custom domain, with graceful handling of 'already exists' responses for idempotent operation.

8.7 Admin Panel Backend

Framework: Express.js with MongoDB. Internal management API for platform operators. Provides endpoints for offering lifecycle management (approve/reject/close), user KYC review and manual override, real-time visibility of all P2P orders (synced via Kafka from the Marketplace service), and platform-wide transaction analytics. Receives live updates from all other services through Kafka consumer subscriptions.

8.8 Transfer Agent Module

Embedded within the Admin Panel Backend. Maintains the complete investor cap table — a real-time record of who holds how many tokens per offering. Provides endpoints for authorizing or rejecting token transfer requests based on compliance rules, generating cap table snapshots for regulatory reporting, and producing complete transfer history audit trails. The Transfer Agent ensures that token ownership records are always consistent with on-chain balances.

8.9 KYC/KYB Compliance Layer

Integrated into the Marketplace Microservice's User Authentication module. Uses the SumSub third-party verification API for both individual investor KYC and institutional KYB (Know Your Business) processes. The kycStatus field on every user record gates access to all sensitive platform operations: primary market investment, P2P listing creation, P2P order placement, and Bonding DEX trading. KYC approval and rejection events trigger automated email notifications through the Notification service.

9. EXPECTED OUTCOMES

The following table summarizes all expected outcomes from the Libertum project and their current delivery status, given that the platform is production-deployed and live:

Table 11: Expected Outcomes and Delivery Status

Expected Outcome	Status
Production-deployed decentralized real estate marketplace on Base blockchain	**ACHIEVED**
Functional P2P order matching engine with smart contract escrow settlement	**ACHIEVED**
Bonding Curve DEX with real-time algorithmically-determined token pricing	**ACHIEVED**
KYC/KYB-gated investment and trading flow using SumSub third-party verification	**ACHIEVED**
Real-time blockchain event tracking with PostgreSQL-based block deduplication	**ACHIEVED**
Multi-tenant SaaS platform with white-label support and plan-based access control	**ACHIEVED**
Automated CI/CD pipeline with Docker containerization and GitHub Actions	**ACHIEVED**
EJS-rendered email notification system covering the full investor lifecycle	**ACHIEVED**

Apache Kafka-based async inter-service messaging across all microservices	**ACHIEVED**
WebSocket real-time browser notifications for order and trade status updates	**ACHIEVED**
Cloudflare automated domain whitelisting for new SaaS tenant activation	**ACHIEVED**
Etherscan API integration for live token holder count enrichment in DEX listings	**ACHIEVED**
High-precision financial arithmetic using Big.js for all monetary calculations	**ACHIEVED**
Transfer Agent module with investor cap table and regulatory audit trail	**ACHIEVED**

Beyond the technical deliverables listed above, the project has produced several qualitative outcomes of significant value:

- **Production Validation:** The platform is not a proof-of-concept or prototype. It operates with real users, real capital, and real blockchain transactions on the Base mainnet, providing production validation of all technical design decisions.
- **Commercial Viability:** The multi-tenant SaaS model demonstrates that the platform's technology can generate recurring revenue through white-label licensing to third-party real estate businesses — establishing a clear commercial path beyond the primary Libertum marketplace.
- **Technical Knowledge Transfer:** The comprehensive microservices architecture, detailed codebase, and deployment documentation serve as a transferable template for future blockchain-integrated financial applications in domains beyond real estate.

10. TIMELINE

The Libertum project is planned across a 9 months development cycle from initial planning through full production deployment. The following Gantt chart details the phasing of all major activities:

Figure 5: Gantt Chart — Libertum Project Development Timeline

The timeline reflects the iterative nature of the development process, with Implementation phases running in parallel across multiple service areas. DevOps infrastructure, testing and deployment activities in months 8-9 represent the final push to production on the Base blockchain mainnet.

11. CONCLUSION

Libertum represents a production-validated, technically rigorous answer to the three core structural problems of traditional real estate investment: illiquidity, inaccessibility, and pricing opacity. Its significance lies not in theoretical possibility but in demonstrated, live production operation serving real users with real capital on the Base blockchain.

11.1 P2P Trading Engine

The P2P Trading Engine delivers a genuinely trustless secondary market where investors can freely enter and exit real estate token positions at any time and at self-defined prices, without relying on brokers, without waiting for property liquidation, and without trusting a centralized counterparty. Behind the scenes, this commercial simplicity is powered by a sophisticated engineering architecture: a seven-state deterministic state machine with atomic MongoDB updates, Kafka-based async synchronization to the Admin service, Promise.all-concurrent email dispatch to both trading parties, real-time WebSocket browser notifications, and Big.js precision arithmetic for all monetary calculations. This is not a demonstration — it is a production-grade trading engine that has processed real trades.

11.2 Bonding Curve DEX

The Bonding Curve DEX provides continuous, transparent, and manipulation-resistant price discovery governed entirely by a mathematical supply-demand function. Beyond basic algorithmic pricing, it delivers comprehensive analytics capabilities that rival professional DeFi platforms: 24-hour volume computation via MongoDB aggregation, time-series chart data with Big.js precision arithmetic, Etherscan-enriched live holder count analytics, and cumulative portfolio value tracking per investor. The linear bonding curve model, calibrated per property offering, provides price appreciation dynamics that naturally mirror real-world property value behaviour — rewarding early investors and creating transparent scarcity-based appreciation as adoption grows.

11.3 Architectural Achievements

The surrounding 8-microservice ecosystem demonstrates production-grade engineering maturity that extends well beyond the two primary modules. Isolated services with strictly no shared databases and all cross-service reads via REST APIs ensure independent deployability and fault isolation. Kafka provides reliable, durable async messaging with guaranteed at-least-once delivery

across all service boundaries. The multi-tenant SaaS platform with automated Cloudflare domain provisioning enables commercial scalability to any number of white-label clients

without manual infrastructure intervention. The Blockchain Event Service's block-number-based deduplication strategy provides exactly-once event processing semantics across arbitrary crash-restart cycles. The fully automated GitHub Actions CI/CD pipeline with manual approval gates ensures zero-downtime deployments.

11.4 Broader Significance

Beyond its immediate technical achievements, Libertum proves a commercially viable thesis that extends beyond real estate: that decentralized finance principles — trustlessness, transparency, permissionless access, algorithmic fairness — can be successfully applied to any traditional illiquid asset class, opening investment opportunities to a global retail audience regardless of geography, capital size, or access to traditional financial infrastructure.

The global RWA tokenization market is projected to reach $16 trillion by 2030. Regulatory frameworks are maturing. Layer 2 blockchain networks have made on-chain transactions economically viable for everyday users. Libertum is production-ready, proven, and architected to scale into this rapidly expanding market. The lessons learned from this internship — in distributed systems engineering, blockchain integration, financial precision arithmetic, and production-grade microservices architecture — represent a foundational competency for the next generation of fintech infrastructure.

11.5 Platform Reliability and Security

Another important achievement of Libertum is platform reliability and operational security. Since the platform handles real investor transactions and blockchain-linked assets, maintaining data consistency and system availability is essential. Authenticated API communication between services, controlled access permissions, and strict backend validation help protect the platform from unauthorized actions and invalid transaction states.

The system also includes structured logging and monitoring, allowing issues to be detected and resolved efficiently. Database-level consistency checks, blockchain event verification, and automated retry mechanisms improve operational stability. These measures ensure that trading activity remains secure and dependable even during high activity or unexpected service interruptions.

11.6 Scalability and Future Readiness

Libertum has been designed with long-term scalability in mind. The microservice architecture allows each service to scale independently based on traffic and processing demand. Core modules such as marketplace trading, token pricing, and blockchain event handling can be upgraded or expanded without affecting the rest of the platform.

The architecture is also flexible enough to support future enhancements including additional blockchain networks, stronger compliance integrations, advanced investor dashboards, and support for new tokenized asset categories. This makes the platform technically prepared for growth as market adoption increases.

11.7 Practical Industry Impact

Libertum also demonstrates strong real-world industry relevance. It bridges the gap between traditional asset ownership and decentralized digital finance by combining compliance-focused tokenization with a user-friendly investment experience. This creates practical value for investors by improving liquidity, increasing accessibility, and offering greater pricing transparency. The project highlights how blockchain can be applied in enterprise financial systems beyond experimentation, delivering measurable benefits through automation, trustless execution, and transparent transaction records.

11.8 Final Outcome

Overall, the internship project successfully delivered both technical and practical outcomes. It provided hands-on experience in blockchain-integrated backend development while contributing directly to a production platform already operating with real users and real investment activity. The successful implementation of the P2P marketplace and Bonding Curve DEX validates the technical feasibility of tokenized real estate trading. Libertum establishes a strong foundation for future innovation in digital asset infrastructure and demonstrates how blockchain technology can transform traditional investment systems into more transparent, liquid, efficient, and globally accessible platforms.

11.9 Technical Learning and Development

A major outcome of this internship was the practical understanding gained through working on a live blockchain-based product. The development process involved applying backend engineering concepts in real scenarios, including service modularization, API integration, asynchronous event handling, database optimization, and blockchain interaction. Working with real production requirements also strengthened problem-solving and debugging skills. Handling token transaction flows, maintaining synchronization between blockchain and backend systems, and implementing scalable business logic provided valuable exposure to modern fintech development practices. This experience helped bridge academic knowledge with real-world software engineering implementation.

11.10 User Experience and Accessibility

Libertum also delivers value through a simplified and accessible user experience. Although the backend system involves blockchain contracts, distributed services, and event-driven communication, the platform presents these complex operations through a straightforward investor workflow.

Users can onboard, browse offerings, purchase fractional ownership, track portfolio value, and participate in P2P trading without needing deep blockchain knowledge. This usability is important because it reduces the technical barrier for adoption and allows a broader investor audience to interact with tokenized assets confidently.

11.11 Business and Commercial Value

From a commercial perspective, Libertum demonstrates a scalable business model for real-world asset tokenization. By digitizing ownership and enabling instant trading, the platform creates new investment opportunities while improving market efficiency.

The white-label multi-tenant SaaS structure also adds significant business value by allowing multiple clients or organizations to launch tokenized investment platforms on shared infrastructure. This improves scalability and supports broader commercial adoption with reduced operational overhead.

11.12 Long-Term Vision

Looking ahead, Libertum provides a strong technical base for future innovation. As blockchain adoption continues to increase and tokenization becomes more widely accepted, platforms like Libertum can play a major role in reshaping investment markets.

The architecture can support expanded features such as automated compliance workflows, cross-chain token support, deeper analytics, and integration with additional financial services. This makes the platform adaptable to evolving industry needs while maintaining its existing production reliability.

11.13 Closing Statement

In conclusion, Libertum successfully demonstrates how blockchain technology, distributed system architecture, and modern backend engineering can be combined to solve real-world financial challenges. The project addresses core investment limitations while maintaining production readiness, technical scalability, and practical usabilit

12. REFERENCES

All references are formatted in IEEE citation style as required by the university evaluation guidelines.

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," Bitcoin.org, 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum Whitepaper, Ethereum Foundation, 2014. [Online]. Available: https://ethereum.org/en/whitepaper/

[3] H. Adams, N. Zinsmeister, and D. Robinson, "Uniswap v2 Core," Uniswap Labs, 2020. [Online]. Available: https://uniswap.org/whitepaper.pdf

[4] S. de la Rouviere, "Tokens 2.0: Curved Token Bonding in Curation Markets," Medium, 2017. [Online]. Available: https://medium.com/@simondlr/tokens-2-0-curved-token-bonding-in-curation-markets-1764a2e0a8a6

[5] Bancor Protocol, "Bancor Protocol: Continuous Liquidity for Cryptographic Tokens through their Smart Contracts," Bancor Whitepaper, 2017. [Online]. Available: https://storage.bancor.network/whitepaper/13651-bancor-protocol-whitepaper-en.pdf

[6] OpenZeppelin, "ERC-20 Token Standard — OpenZeppelin Documentation," 2023. [Online]. Available: https://docs.openzeppelin.com/contracts/4.x/erc20

[7] ERC-1404 Authors, "ERC-1404: Simple Restricted Token Standard," Ethereum Improvement Proposals, 2018. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1404

[8] T. Hardjono, A. Lipton, and A. Pentland, "Toward an Interoperability Architecture for Blockchain Autonomous Systems," IEEE Transactions on Engineering Management, vol. 67, no. 4, pp. 1298–1309, Nov. 2020.

[9] M. Conti, E. S. Kumar, C. Lal, and S. Ruj, "A Survey on Security and Privacy Issues of Bitcoin," IEEE Communications Surveys & Tutorials, vol. 20, no. 4, pp. 3416–3452, 2018.

[10] RealT LLC, "Real Estate Tokenization on Ethereum," RealT Platform, 2020. [Online]. Available: https://realt.co

[11] Base Protocol, "Base: An Ethereum L2 Built on the OP Stack," Coinbase, 2023. [Online]. Available: https://base.org

[12] OpenSea, "OpenSea Developer Documentation — NFT Marketplace Protocol," 2022. [Online]. Available: https://docs.opensea.io

[13] NestJS, "NestJS — A Progressive Node.js Framework," NestJS Documentation, 2023. [Online]. Available: https://docs.nestjs.com

[14] Apache Software Foundation, "Apache Kafka: A Distributed Streaming Platform," 2023. [Online]. Available: https://kafka.apache.org/documentation/

[15] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. O'Reilly Media, 2021.

[16] M. Kleppmann Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, 2017.

[17] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Doctoral Dissertation, University of California, Irvine, 2000.

[18] A. M. Antonopoulos and G. Wood, Mastering Ethereum: Building Smart Contracts and DApps. O'Reilly Media, 2018.

[19] Boston Consulting Group, "Relevance of On-Chain Asset Tokenization in 'Crypto Winter'," BCG & ADDX, 2022. [Online]. Available: https://www.bcg.com

[20] D. Yermack, "Corporate Governance and Blockchains," Review of Finance, vol. 21, no. 1, pp. 7–31, 2017.

[21] Ethereum Foundation, "Ethereum Whitepaper," 2014. [Online]. Available: https://ethereum.org/en/whitepaper/

[22] G. Wood, *Ethereum: A Secure Decentralised Generalised Transaction Ledger*, Ethereum Project Yellow Paper, 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[23] MongoDB Inc., "MongoDB Documentation," 2023. [Online]. Available: https://www.mongodb.com/docs/

[24] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2023. [Online]. Available: https://www.postgresql.org/docs/

[25] Cloudflare Inc., "Cloudflare Developer Documentation," 2023. [Online]. Available: https://developers.cloudflare.com/

[26] GitHub Inc., "GitHub Actions Documentation," 2023. [Online]. Available: https://docs.github.com/en/actions

[27] WebSocket.org, "The WebSocket Protocol Explained," 2023. [Online]. Available: https://websocket.org/

[28] M. Corbic, "Big.js: Arbitrary-Precision Decimal Arithmetic for JavaScript," GitHub Repository, 2023. [Online]. Available: https://github.com/MikeMcl/big.js/

[29] Coinbase Developer Platform, "Base Developer Documentation," 2023. [Online]. Available: https://docs.base.org/

[30] Chainlink Labs, "Blockchain Oracles and Smart Contract Integration," 2023. [Online]. Available: https://chain.link/education

[31] P. Hunt, "Event-Driven Architecture Patterns for Scalable Systems," *IEEE Software*, vol. 37, no. 4, pp. 52–60, 2020.

[32] Docker Inc., "Docker Documentation," 2023. [Online]. Available: https://docs.docker.com/

[33] Redis Labs, "Redis Documentation," 2023. [Online]. Available: https://redis.io/docs/

[34] OpenAPI Initiative, "OpenAPI Specification," 2023. [Online]. Available: https://swagger.io/specification/

[35] Deloitte, "Blockchain and Real Estate: The Future of Property Investment," Deloitte Insights, 2022. [Online]. Available: https://www2.deloitte.com/

[36] PwC, "Asset Tokenization and the Future of Financial Markets," PwC Global Report, 2022. [Online]. Available: https://www.pwc.com/

[37] World Economic Forum, "Asset Tokenization in Financial Services," WEF Report, 2023. [Online]. Available: https://www.weforum.org/

[38] M. Swan, *Blockchain: Blueprint for a New Economy*. Sebastopol, CA, USA: O'Reilly Media, 2015.

[39] J. Poon and T. Dryja, "The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments," 2016. [Online]. Available: https://lightning.network/lightning-network-paper.pdf

[40] K. Werbach, *The Blockchain and the New Architecture of Trust*. Cambridge, MA, USA: MIT Press, 2018.

[42] MongoDB Inc., "MongoDB Documentation," 2023. [Online]. Available: https://www.mongodb.com/docs/

[43] Hardhat Foundation, "Hardhat Documentation," 2023. [Online]. Available: https://hardhat.org/docs

[44] Foundry Contributors, "Foundry Book — Ethereum Development Toolkit," 2023. [Online]. Available: https://book.getfoundry.sh/

[45] Cloudflare Inc., "Cloudflare Developer Documentation," 2023. [Online]. Available: https://developers.cloudflare.com/

[46] Ethereum Foundation, "Ethereum Improvement Proposals (EIPs)," 2023. [Online]. Available: https://eips.ethereum.org/

[47] Polygon Technology, "Polygon zkEVM Documentation," 2023. [Online]. Available: https://docs.polygon.technology/zkEVM/

[48] Arbitrum Foundation, "Arbitrum Developer Documentation," 2023. [Online]. Available: https://docs.arbitrum.io/

[49] OpenZeppelin, "OpenZeppelin Contracts Documentation — Smart Contract Security Library," 2023. [Online]. Available: https://docs.openzeppelin.com/contracts/

[50] Kubernetes Authors, "Kubernetes Documentation — Production-Grade Container Orchestration," 2023. [Online]. Available: https://kubernetes.io/docs/

13. APPENDIX

A. Implementation Screenshots

The following figures contain screenshots from the live Libertum platform deployed on the Base blockchain. These images demonstrate the platform's production functionality across its major user interfaces.

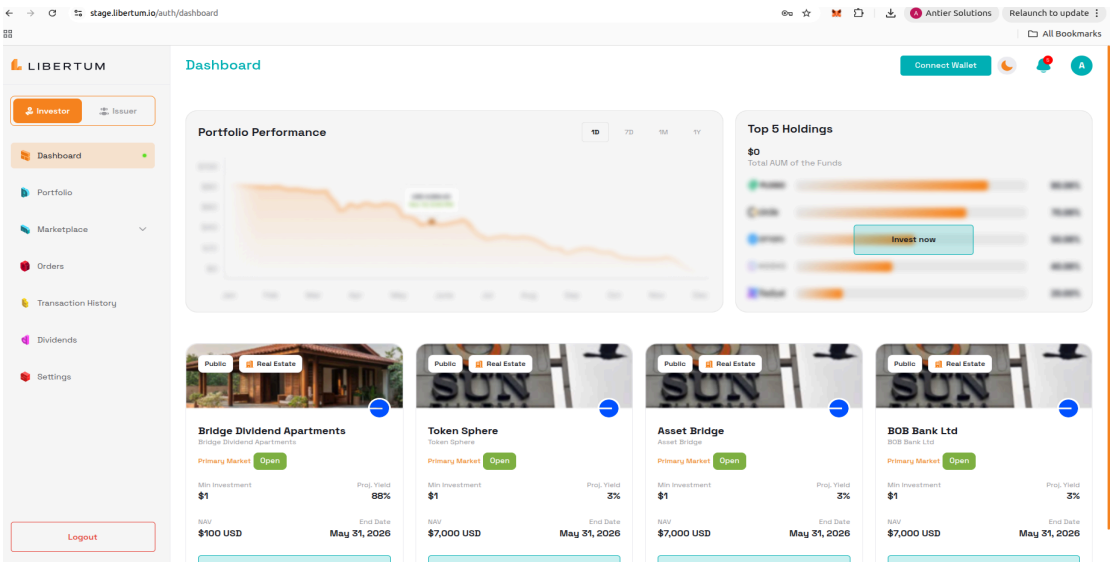

Figure 6: Libertum Investor Dashboard — Showing Offering Listings, Investors, Portfolio Performance, and Top Holdings

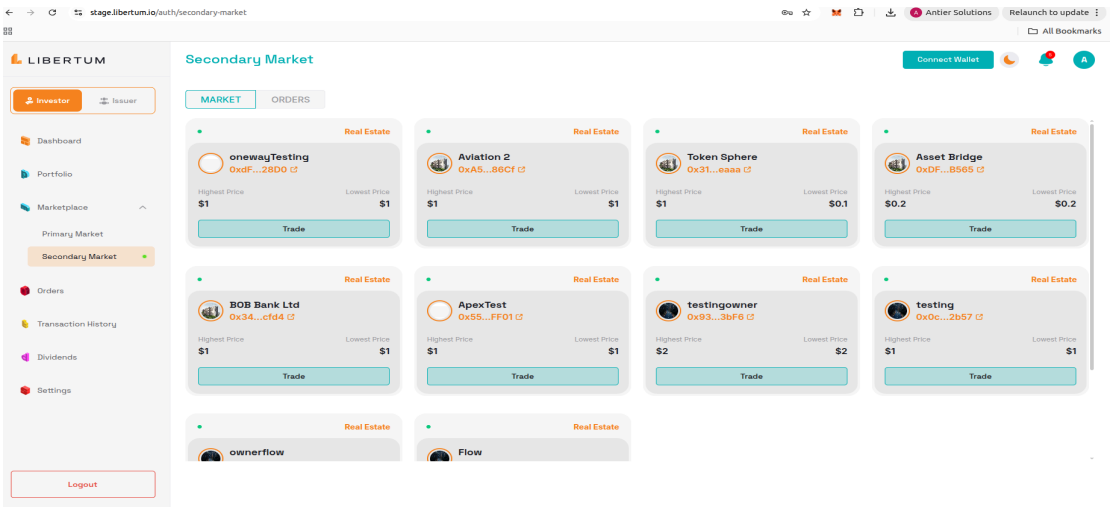

Figure 7: Libertum Secondary Market(P2P) Pannel — Showing tokenized real estate asset listings, live pricing information, investor trade options, and marketplace order execution interface for secondary trading

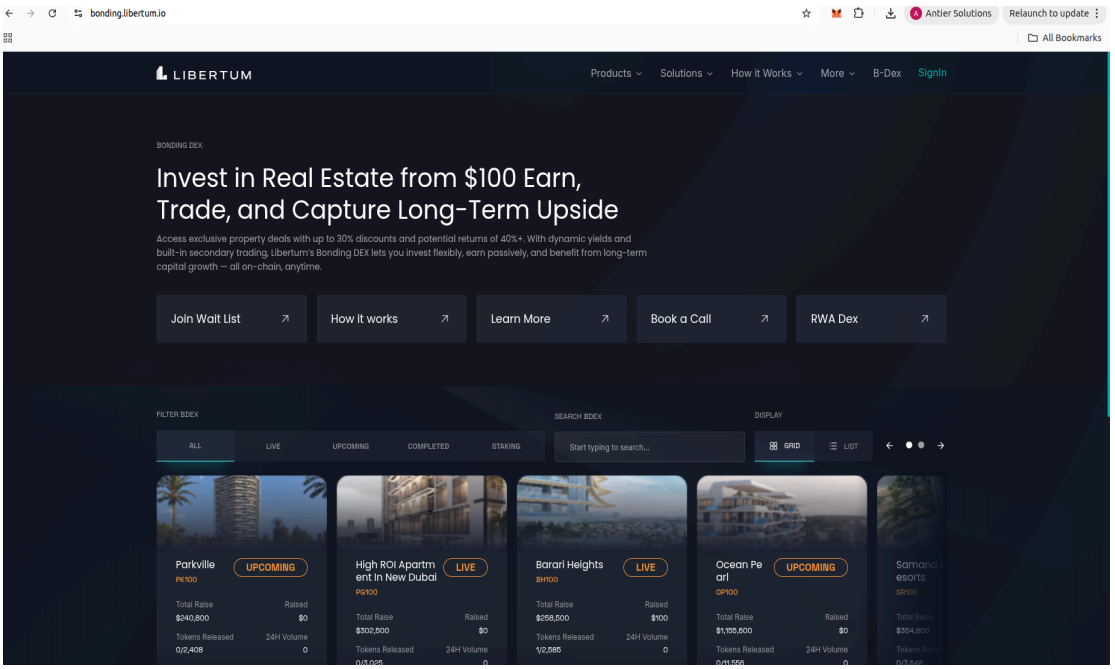

Figure 8: Bonding Dex Platform

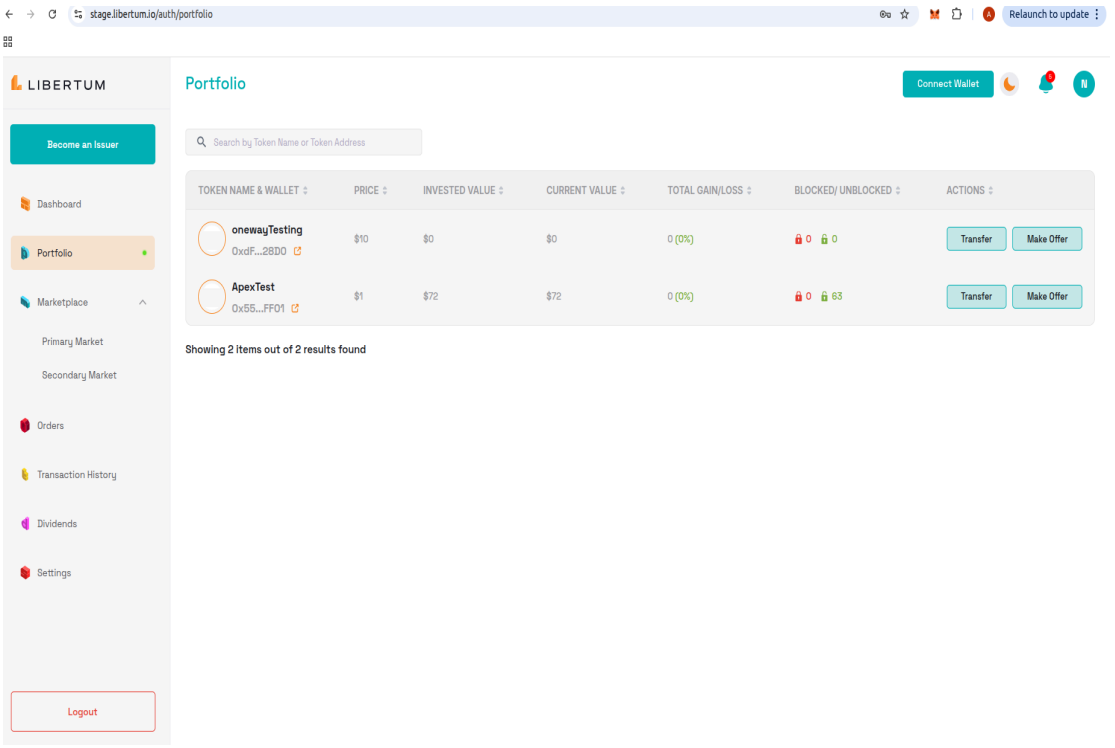

Figure 9: Investors Portfolio (P2P initiation)

B. Key Code Snippets

Snippet 1: Bonding Curve Price Calculation (TypeScript)

```
// Linear Bonding Curve: Price = BasePrice * (1 + k * supply)
function calculateBondingPrice(
    basePrice: string,
    k: string,
    supply: string
): string {
    const bp           = new Big(basePrice);
    const curveK       = new Big(k);
    const currentSupply = new Big(supply);
    return bp.times(
        new Big(1).plus(curveK.times(currentSupply))
    ).toFixed(6);
}
```

Snippet 2: P2P Order Quantity Deduction (Atomic MongoDB Update)

```
// Atomically deduct ordered qty from listing availableQuantity
await P2PListingSchema.findOneAndUpdate(
    { _id: listingId, availableQuantity: { $gte: requestedQty } },
    { $inc: { availableQuantity: -requestedQty } },
    { new: true }
);
```

C. Glossary of Terms

Table 13: Glossary of Technical Terms and Acronyms

Term / Acronym	Definition
AMM	Automated Market Maker — a type of DEX that uses mathematical formulas to price assets.
Bonding Curve	A mathematical function defining the relationship between a token's supply and its price.
Base Blockchain	An Ethereum-compatible Layer 2 blockchain built by Coinbase on the OP Stack.
Bull / BullMQ	Node.js job queue libraries built on Redis, used for background job scheduling.
Cap Table	Capitalization table — a record of who owns what proportion of a company's shares or tokens.

DEX	Decentralized Exchange — a peer-to-peer trading marketplace without a central authority.
DeFi	Decentralized Finance — financial services built on blockchain smart contracts.
ERC-20	Ethereum token standard for fungible (interchangeable) digital tokens.
ERC-1155	Multi-token Ethereum standard allowing a single contract to manage multiple token types.
EVM	Ethereum Virtual Machine — the computation environment for Ethereum smart contracts.
JWT	JSON Web Token — a compact, URL-safe means of representing claims for authentication.
Kafka	Apache Kafka — a distributed event streaming platform for high-throughput messaging.
KYB	Know Your Business — institutional identity verification for business entities.
KYC	Know Your Customer — individual identity verification for regulatory compliance.
P2P	Peer-to-Peer — direct interaction between participants without intermediaries.
RWA	Real-World Asset — physical or traditional financial assets represented on blockchain.
Solidity	The programming language for writing Ethereum smart contracts.
Tokenization	The process of representing ownership of real-world assets as blockchain tokens.
WebSocket	A communication protocol providing full-duplex channels over a single TCP connection.